Project Documentation

How to Use This Document

This document is the single source of truth for this project.
It is generated automatically from the `docs/` folder and updated whenever a module completes.

Who should read what:

Section	Audience	Purpose
Business	Everyone	Understand the problem being solved and the business rules
Requirements	PM, Tech Lead	What the system must do
Architecture	Engineers, Tech Lead	How the system is structured and deployed
Specifications	Engineers	Schema, API contracts, permissions, logging
Flows	Engineers	Code-level execution flows per module
Project Status	Tech Lead	What has been built and how code is organized

Diagrams: Static images are embedded in each section.
Click "Open interactive version" to open the live draggable/zoomable version in your browser.

To regenerate this PDF:

```
python3 docs/script/build_pdf.py docs --lang en -o docs/project-documentation-en.pdf
```

Table of Contents

- API Contract
- Permissions
- Pipeline Contract
- CLI Contract
- Public API
- Model Contract
- Service Catalog
- Service Contract
- LLM Contract
- prompt-library
- RAG Contract
- MCP Contract
- Mobile Contract
- **4. Build**
- Codebase Map
- Dependencies
- architecture/deployment.md
- Experiment Log
- Evaluation Spec
- **5. Test**
- Test Plan
- Test Report
- Eval Log
- **6. Deployment**
- logging-spec.md
- Quickstart
- Release Guide
- Compatibility Matrix
- Runbook
- Drift Policy

1. Introduction

Project Requirements

Goals

- [Primary goal]
- [Secondary goal]

Scope

In Scope

- [Included features or areas]

Out of Scope

- [Excluded features or areas]

Roles

Role	Description

Business Processes

- **[Process name]**: [Brief description of what this process does]
- **[Process name]**: [Brief description]

Functional Requirements

- **FR-001**: [What the system MUST do]
- **FR-002**: [What the system MUST do]
- **FR-003**: [NEEDS CLARIFICATION: describe what is unclear]

Non-Functional Requirements

- **Performance**: [e.g., Web App: p95 < 200ms | CLI: execution < 5s | Pipeline: 1M rows in < 10min | LLM App: first token < 2s]
- **Availability**: [e.g., 99.9% uptime | N/A for CLI Tool / Library]
- **Security**: [e.g., Web App: JWT on all endpoints | Pipeline: encrypted credentials | LLM App: no PII in prompts]
- **Scalability**: [e.g., Web App: 10,000 concurrent users | Pipeline: 100M rows per run | LLM App: 50 concurrent sessions]

Edge Cases

Empty and missing input

- [Scenario] → [Expected behaviour]

Permission boundaries

- [Scenario] → [Expected behaviour]
- *(Skip if project type has no auth — e.g., CLI Tool, Library, Data Pipeline)*

Concurrency and race conditions

- [Scenario] → [Expected behaviour]

External dependency failures

- [Scenario] → [Expected behaviour]
- *(For AI / LLM App: include LLM API timeout and content filter block)*

State machine violations

- [Scenario] → [Expected behaviour]
- *(Skip if project has no state machine — e.g., Library, CLI Tool, AI / LLM App)*

Data contract violations

- [Scenario] → [Expected behaviour]
- *(Applies to Data Pipeline, ML Pipeline, AI / LLM App with RAG)*

Acceptance Criteria

- **AC-001**: Given [initial state], When [action], Then [expected result]

- **AC-002**: Given [initial state], When [action], Then [expected result]

Assumptions

- [Assumption]
- [Assumption]

business-rules

Approval Rules

Action	Required approver	Trigger	Rejection response
[e.g., Role change]	Admin	[e.g., POST /api/roles / admin CLI command]	[e.g., 403 / error message]

Validation Rules

Rule	Condition checked	Failure behavior
[e.g., Report date range]	`from ≤ to`	400 before DB query

Notification Rules

When	Who receives	Method

Audit Rules

Action	What is retained

Glossary

Business Terms

Term	Definition

Technical Terms

Term	Definition

Abbreviations

Abbreviation	Full form	Context

2. Plan

Project Plan

Sprint 1: Shared Foundation

Tasks in this sprint:
- Task 1: INF [Foundation Name]

Task 1: INF [Foundation Name]

Goal: [What this task achieves]

Context: `[file path]` — [why it needs to be read before starting]

Files:
- Create: `[file path]`
- Modify: `[file path]`

Doc Checklist:

- [ ] `docs/[relevant spec]` — [what to check]
- [ ] **Step 1: [Step name]**
  [What to do. Expected result: [description]]
- [ ] **Step 2: [Step name]**
  [What to do. Expected result: [description]]
- [ ] **Verify**
  Run: `[exact command]`
  Expected: `[exact output or behaviour]`
  Do not mark this task complete until the expected output is confirmed.

Sprint 2: [Feature A]

Tasks in this sprint:
- Task 2: DB [Feature A] Schema
- Task 3: BE [Feature A]
- Task 4: FE [Feature A]

Task 2: DB [Feature A] Schema

Goal: [Description]

Context: `[schema file]` — [why]

Files:
- Create: `[migration file path]`

Doc Checklist:
- [ ] `docs/specs/data-model.md` — update schema, indexes, state machine if changed
- [ ] `docs/architecture/database.md` — update if main entities or relationships changed

- [ ] **Step 1: [Step name]**
  [Description. Expected result: [description]]

- [ ] **Verify**
  Run: `[exact command]`
  Expected: `[exact output]`
  Do not mark this task complete until the expected output is confirmed.

Task 3: BE [Feature A]

Goal: [Description]

Context: `[file path]` — [why]

Files:
- Create: `[file path]`
- Modify: `[file path]`

Doc Checklist:
- [ ] `docs/specs/api-contract.md` — update if endpoints or error codes changed
- [ ] `docs/specs/permissions.md` — update if roles or endpoint access changed
- [ ] `docs/specs/logging-spec.md` — add module name if new module introduced
- [ ] `docs/business/business-rules.md` — update if business constraints changed

- [ ] **Step 1: [Step name]**
  [Description. Expected result: [description]]

- [ ] **Verify**
  Run: `[exact command]`
  Expected: `[exact output]`
  Do not mark this task complete until the expected output is confirmed.

Task 4: FE [Feature A]

Goal: [Description]

Context: `[file path]` — [why]

Files:
- Create: `[file path]`

Doc Checklist:
- [ ] `docs/architecture/frontend.md` — update if page structure or component strategy changed
- [ ] `docs/codebase-map.md` (page structure block) — update if new pages/screens added

- [ ] **Step 1: [Step name]**
  [Description]

- [ ] **Verify**
  Run: `[exact command or manual step]`
  Expected: `[exact output or behaviour]`
  Do not mark this task complete until the expected output is confirmed.

Sprint 3: [Feature B]

Tasks in this sprint:
- Task 5: DB [Feature B] Schema
- Task 6: BE [Feature B]
- Task 7: FE [Feature B]

Task 5: DB [Feature B] Schema

Goal: [Description]

Context: `[file path]` — [why]

Files:
- Create: `[file path]`

Doc Checklist:
- [ ] `docs/specs/data-model.md` — update schema, indexes, state machine if changed
- [ ] `docs/architecture/database.md` — update if main entities or relationships changed

- [ ] **Step 1: [Step name]**
  [Description]

- [ ] **Verify**
  Run: `[exact command]`

Expected: `[exact output]`
Do not mark this task complete until the expected output is confirmed.

Task 6: BE [Feature B]

Goal: [Description]

Context: `[file path]` — [why]

Files:
- Create: `[file path]`

Doc Checklist:
- [ ] `docs/specs/api-contract.md` — update if endpoints or error codes changed
- [ ] `docs/specs/permissions.md` — update if roles or endpoint access changed
- [ ] `docs/specs/logging-spec.md` — add module name if new module introduced
- [ ] `docs/business/business-rules.md` — update if business constraints changed

- [ ] **Step 1: [Step name]**
  [Description]

- [ ] **Verify**
  Run: `[exact command]`
  Expected: `[exact output]`
  Do not mark this task complete until the expected output is confirmed.

Task 7: FE [Feature B]

Goal: [Description]

Context: `[file path]` — [why]

Files:
- Create: `[file path]`

Doc Checklist:
- [ ] `docs/architecture/frontend.md` — update if page structure or component strategy changed
- [ ] `docs/codebase-map.md` (page structure block) — update if new pages/screens added

- [ ] **Step 1: [Step name]**
  [Description]

- [ ] **Verify**
  Run: `[exact command or manual step]`
  Expected: `[exact output or behaviour]`
  Do not mark this task complete until the expected output is confirmed.

Completed

- [Task name] — [completion date]

Changelog

---

3. Design

Architecture

Overview

[Brief description of the system architecture, covering main components and core data flow. 2-4 sentences.]

System Components

Component	Type	Responsibility	Protocol

Data Flow

Main Path

1. [Component A] → [Component B]: [description]
2. [Component B] → [Component C]: [description]

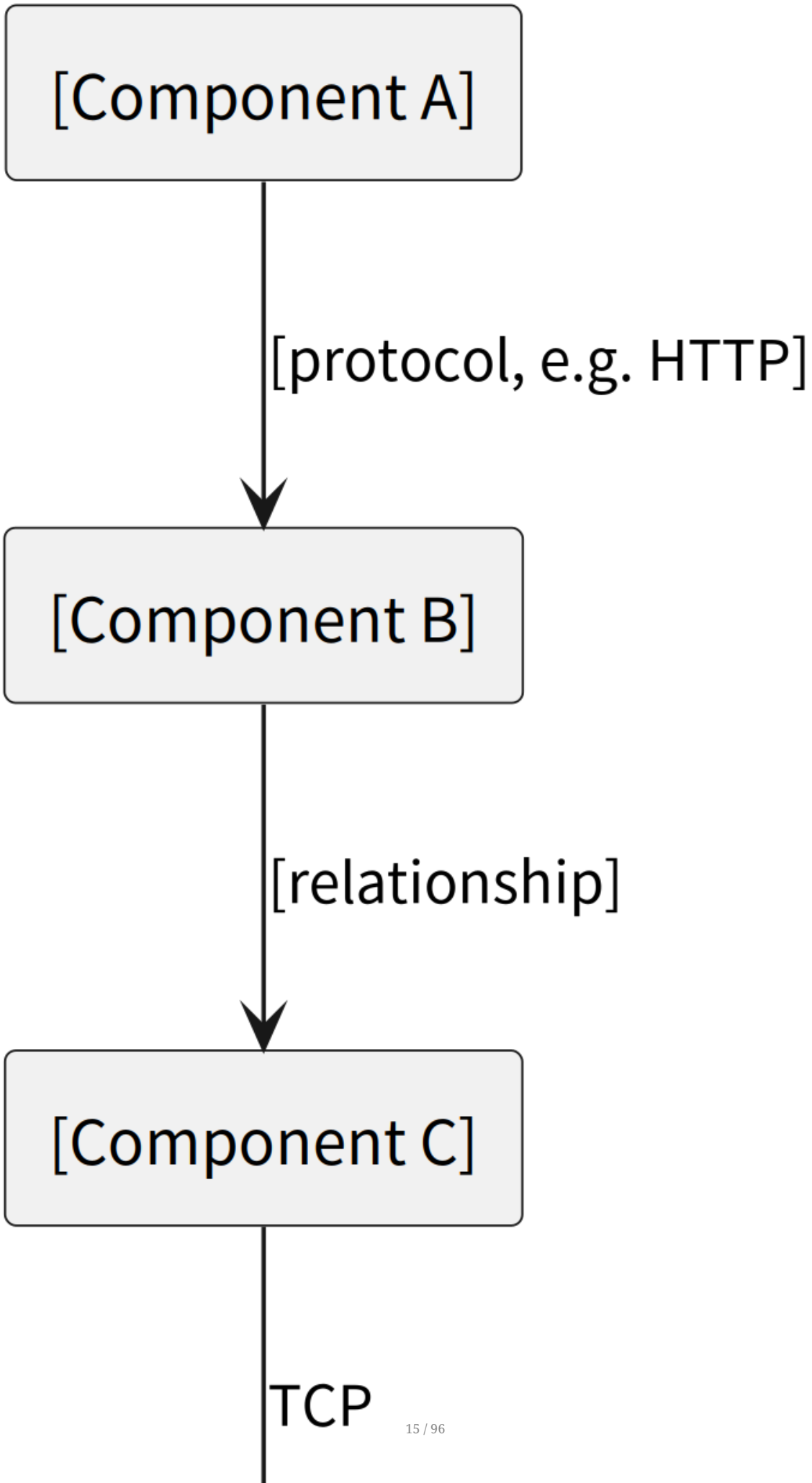
Async Path

1. [Component A] → [Queue / Event Bus]: [event sent]
2. [Queue / Event Bus] → [Component B]: [event consumed]

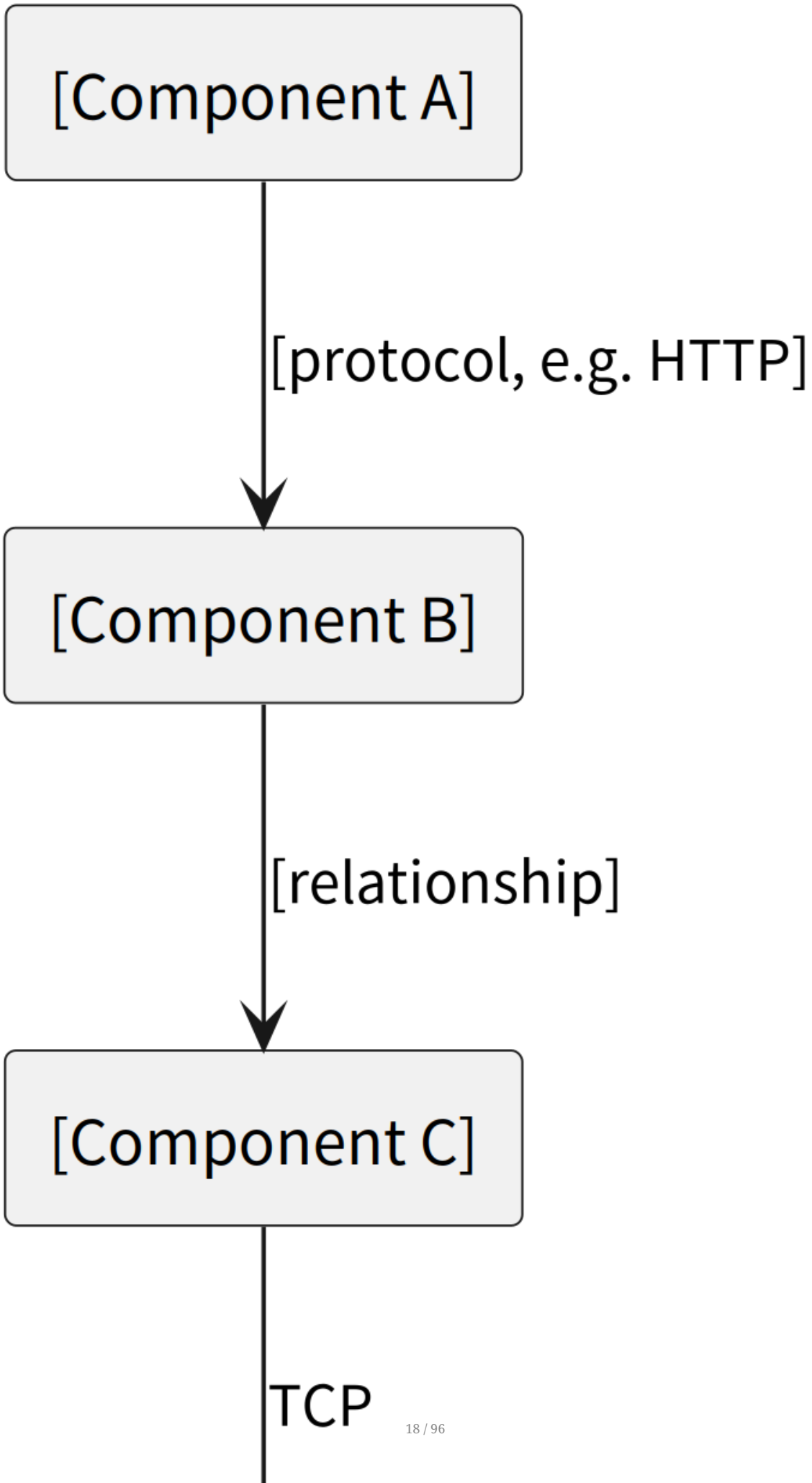
Error Path

1. [error scenario] → [how the component handles it]

System Architecture Diagram


[Component A]
[protocol, e.g. HTTP]
[Component B]
[relationship]
[Component C]
TCP

System Component Structure


[Component A]
[protocol, e.g. HTTP]
[Component B]
[relationship]
[Component C]
TCP

architecture/backend.md

> **Applies to:** Web App, Microservices, CLI Tool (if layered), AI/LLM App (server component). Data Pipelines and ML Pipelines document their processing layers in `pipeline-contract.md`. Libraries/SDKs document their public API surface in `public-api.md`.

Purpose:
Describe backend / processing structure — what stack is used, how it is layered, and how modules are organized.

Include:
- Stack
- Layering (use your actual layer names and count — do not assume any fixed pattern)
- Layer responsibilities
- Module pattern (file structure per feature)

Avoid:
- Full API endpoint list
- Feature requirements
- Business rules

Stack

[List the runtime, framework, data access layer, and storage. Examples by type:

Web App / Microservices:
Node.js 22 / Express / Prisma / PostgreSQL
Python 3.12 / FastAPI / SQLAlchemy / PostgreSQL
Go 1.22 / Gin / sqlx / MySQL
Java 21 / Spring Boot / JPA / PostgreSQL
Ruby 3 / Rails / ActiveRecord / PostgreSQL

CLI Tool:
Python 3.12 / Click / — / (no DB, or SQLite for local state)
Go 1.22 / cobra / — / (no DB)
Node.js 22 / commander / — / (no DB)

AI / LLM App (server component):
Python 3.12 / FastAPI / LangChain or LlamaIndex / Chroma + PostgreSQL]

Layering

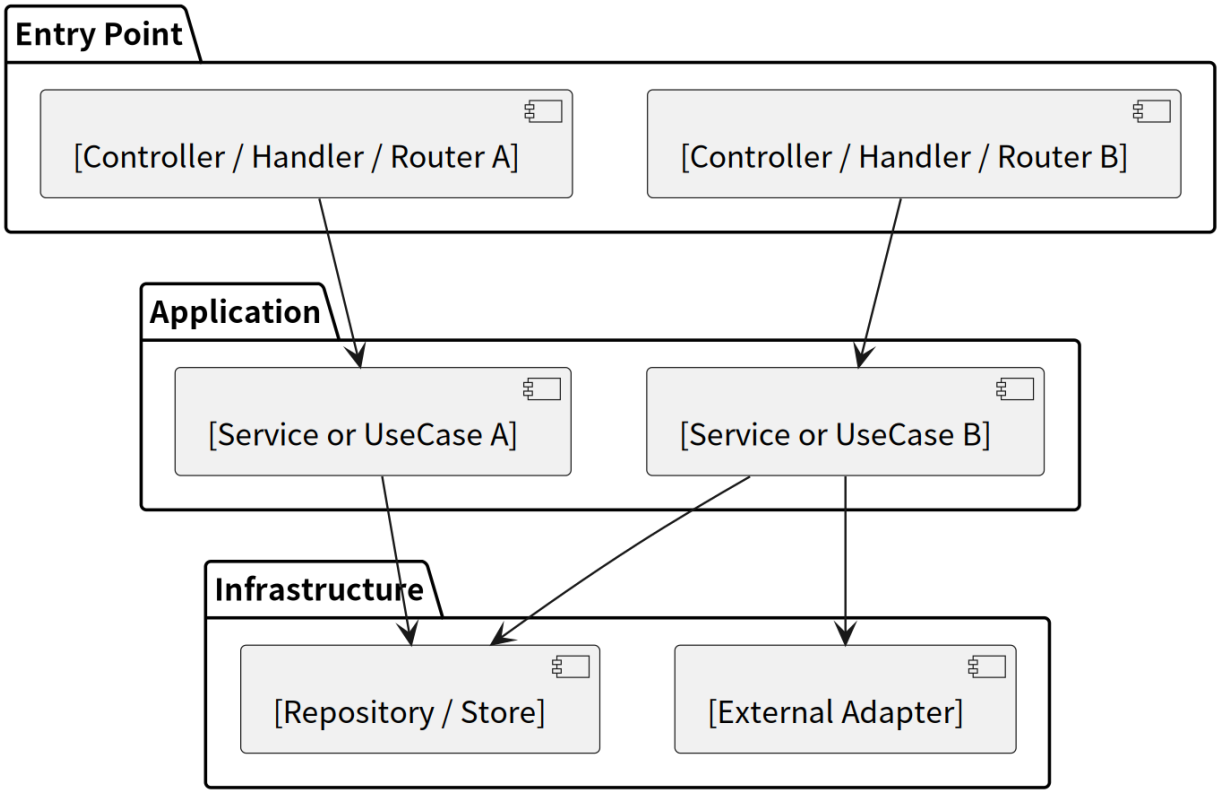
```
[Layer 1] → [Layer 2] → [Layer 3] → ...
```

Layer	Responsibility

Module Pattern

```
[show actual file tree for one representative module]
```

Component Structure



architecture/frontend.md

> **Applies to:** Web App, Microservices (with a UI layer), Mobile App.
> If your project has no frontend (CLI Tool, Library/SDK, Data Pipeline, ML Pipeline, AI/LLM App script), **skip this file entirely** — delete it from your project's docs/architecture/ folder.

Purpose:
Describe frontend structure — what stack is used, how pages and components are organized, and how data fetching is handled.

Include:
- Stack
- Page / screen structure
- Component strategy
- Data fetching / state management strategy
- Shared UI standards

Avoid:
- Page-by-page requirements
- UI text
- Business workflow details

Stack

[List the framework, language, styling approach, and data fetching library. e.g.:
React 18 / TypeScript / Tailwind CSS / React Query
Vue 3 / TypeScript / Pinia / Axios
Next.js 14 / TypeScript / Tailwind CSS / SWR
Svelte / TypeScript / TailwindCSS
Flutter / Dart
Swift / SwiftUI
Android / Kotlin / Jetpack Compose
Angular / TypeScript / RxJS]

Page / Screen Structure

```
[show actual folder structure for one representative feature]
```

Component Strategy

[Describe component split principles]

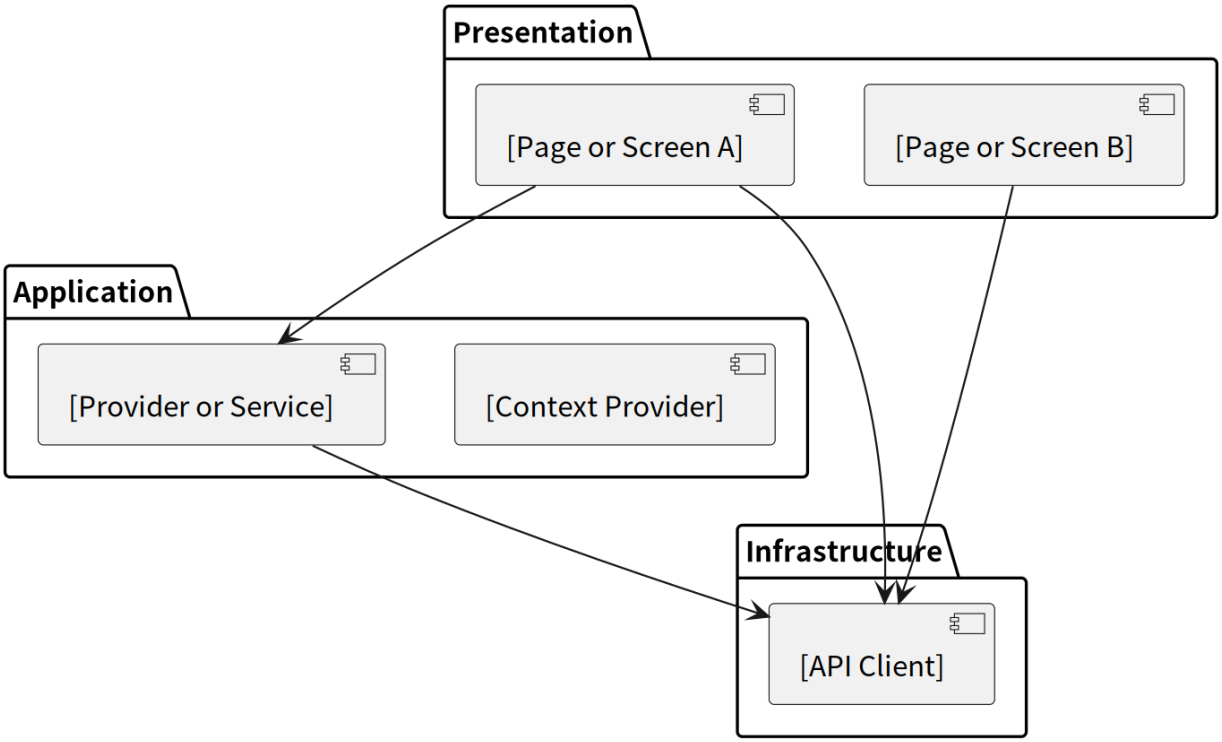
Data Fetching / State Management

[Describe the data fetching and state management approach used in this project]

Shared UI Standards

[Describe shared components, design system, icon library, form library, etc.]

Component Structure



Mobile App Variant

Platform & Framework

Property	Value
Framework	[React Native / Flutter / SwiftUI / Jetpack Compose]
Language	[TypeScript / Dart / Swift / Kotlin]
Platforms	[iOS / Android / both]
Styling	[StyleSheet / NativeWind / styled-components / Tailwind (RN) / Theme system (Flutter)]
Navigation	[React Navigation / Expo Router / Flutter Navigator 2.0 / UIKit / Jetpack Navigation]
State management	[Redux Toolkit / Zustand / Riverpod / Provider / MobX / ViewModel + StateFlow]

Screen Structure

```
[show actual folder structure for one representative feature / screen group]
```

Component / Widget Strategy

[Describe component / widget split principles]

Platform-Specific Adaptations

Feature	iOS behaviour	Android behaviour
[Push permission]	Must explicitly request with prompt	Granted by default (Android 12−) / prompted (Android 13+)
[Biometric auth]	Face ID / Touch ID via LocalAuthentication	Fingerprint / Face via BiometricPrompt

architecture/database.md

Purpose:
Describe the database at a conceptual level — what exists, how it relates, and why it was designed this way. Field-level detail belongs in docs/specs/data-model.md.

Include:
- Database engine
- Main entities (3-5 sentences, no field lists)
- Key relationships
- Important constraints
- Schema decisions (why, not what)

Avoid:
- Full schema duplication
- Field-by-field listings
- API or UI details

Database Engine

[Database engine and version]

Main Entities

[Description of core entities / collections / nodes and how they relate]

Key Relationships

- [Entity / Collection A] → [Entity / Collection B]: [relationship type and reason]
- [Entity / Collection A] → [Entity / Collection C]: [relationship type and reason]

Important Constraints

- [Constraint and the business rule it enforces]

Schema Decisions

- [Decision and rationale]

Distribution

Package Identity

Property	Value
Package name	[e.g., `my-library`, `my-cli`]
Registry	[npm / PyPI / crates.io / GitHub Releases / Homebrew / etc.]
Registry URL	[Direct link to the package page]
Source repository	[GitHub / GitLab URL]

Build

Prerequisites:

```
# Install build tools
[command — e.g., npm install / pip install build / cargo build]
```

Build command:

```
[build command — e.g., npm run build / python -m build / cargo build --release]
```

Output:

Artifact	Path	Description
[Library bundle / Binary]	`[dist/index.js / dist/*.whl / target/release/binary]`	[What it is]
[Type declarations]	`[dist/index.d.ts]`	[TypeScript types — if applicable]

Publish

Authentication:

```
# Authenticate with the registry before publishing
[auth command — e.g., npm login / twine upload requires PYPI_TOKEN env var]
```

Publish command:

```
[publish command — e.g., npm publish / twine upload dist/* / cargo publish]
```

Required before publishing:

- Version bumped in `[pyproject.toml / package.json / Cargo.toml]`
- `CHANGELOG.md` updated (see `docs/specs/release-guide.md`)
- All tests passing
- Git tag created and pushed

Installation (end user)

```
# npm
npm install [package-name]

# PyPI
pip install [package-name]

# Homebrew (if applicable)
brew install [formula-name]

# Binary download (if applicable)
curl -L [release URL] | tar xz
```

CI/CD Pipeline

Stage	Trigger	Action
Test	Every PR and push	Run full test suite
Build	Every merge to `main`	Build distributable artifact
Publish	Git tag `v*` pushed	Publish to registry

CI configuration: [`[link to CI config — .github/workflows/*.yml / .gitlab-ci.yml]`]

Release Artifacts

For each GitHub/GitLab release, attach:

File	Description
`[package-name]-[version].tar.gz`	Source archive
`[binary-name]-linux-x86_64`	Linux binary (if CLI)
`[binary-name]-darwin-arm64`	macOS ARM binary (if CLI)
`[binary-name]-windows-x86_64.exe`	Windows binary (if CLI)

Checksums (`SHA256SUMS`) must be attached alongside binaries.

Mobile App Variant

App Identity

Property	iOS	Android
Bundle ID	`com.example.myapp`	`com.example.myapp`
App Store / Play Store ID	[App Store ID]	[Play Store package name]
Version naming	`[major].[minor].[patch]`	`[major].[minor].[patch]`
Build number	Auto-incremented by CI	`versionCode` auto-incremented by CI

Build Pipeline

Tool	Purpose
[Fastlane / Xcode Cloud / Bitrise / GitHub Actions]	CI orchestration
[Xcode / Gradle]	Platform build toolchain
[EAS Build / Expo]	Cross-platform build service (if using Expo)

iOS build command:

```
# Local (ad-hoc / TestFlight)
fastlane ios beta

# Or via Xcode
xcodebuild -workspace [App.xcworkspace] -scheme [App] -configuration Release \
  -archivePath build/App.xcarchive archive
```

Android build command:

```
# Local APK
./gradlew assembleRelease

# AAB for Play Store
./gradlew bundleRelease
```

Signing Configuration

iOS:

Artifact	Certificate	Profile
Development	Apple Development	`[Team] Development`
TestFlight	Apple Distribution	`[App] App Store`
App Store	Apple Distribution	`[App] App Store`

Certificates and profiles are stored in [Keychain / Match / manual download].
Do NOT commit `.p12` files or provisioning profiles to git.

Android:

Property	Value
Keystore file	`upload-keystore.jks` — stored in [secure vault, never in git]
Key alias	`[upload]`
Credentials source	CI environment variables: `KEYSTORE_BASE64`, `KEY_ALIAS`, `KEY_PASSWORD`, `STORE_PASSWORD`

Release Checklist

Before each App Store / Google Play release:

- [ ] Version and build number bumped in `[package.json / pubspec.yaml / Info.plist / build.gradle]`

- [ ] Release notes written for this version
- [ ] All CI tests passing on `main`
- [ ] Signed build generated and uploaded to TestFlight / Internal Testing track
- [ ] QA sign-off on TestFlight / Internal Testing build
- [ ] App Store / Play Store metadata updated (screenshots, description if changed)
- [ ] Compliance questionnaire answered (IDFA, encryption export)

Submission:

```
# iOS — upload to App Store Connect
fastlane ios release

# Android — upload AAB to Play Console
fastlane android release
```

CI/CD Pipeline

Stage	Trigger	Action
Test	Every PR	Unit + integration tests, lint
Build (dev)	Merge to `develop`	Debug build, upload to TestFlight Internal / Play Internal
Build (release)	Merge to `main` or git tag `v*`	Release build, upload to TestFlight / Play Store
Submit	Manual trigger	Submit to App Review / Production track

CI configuration: [`[link to CI config]`]

Infrastructure Topology

Resource Inventory

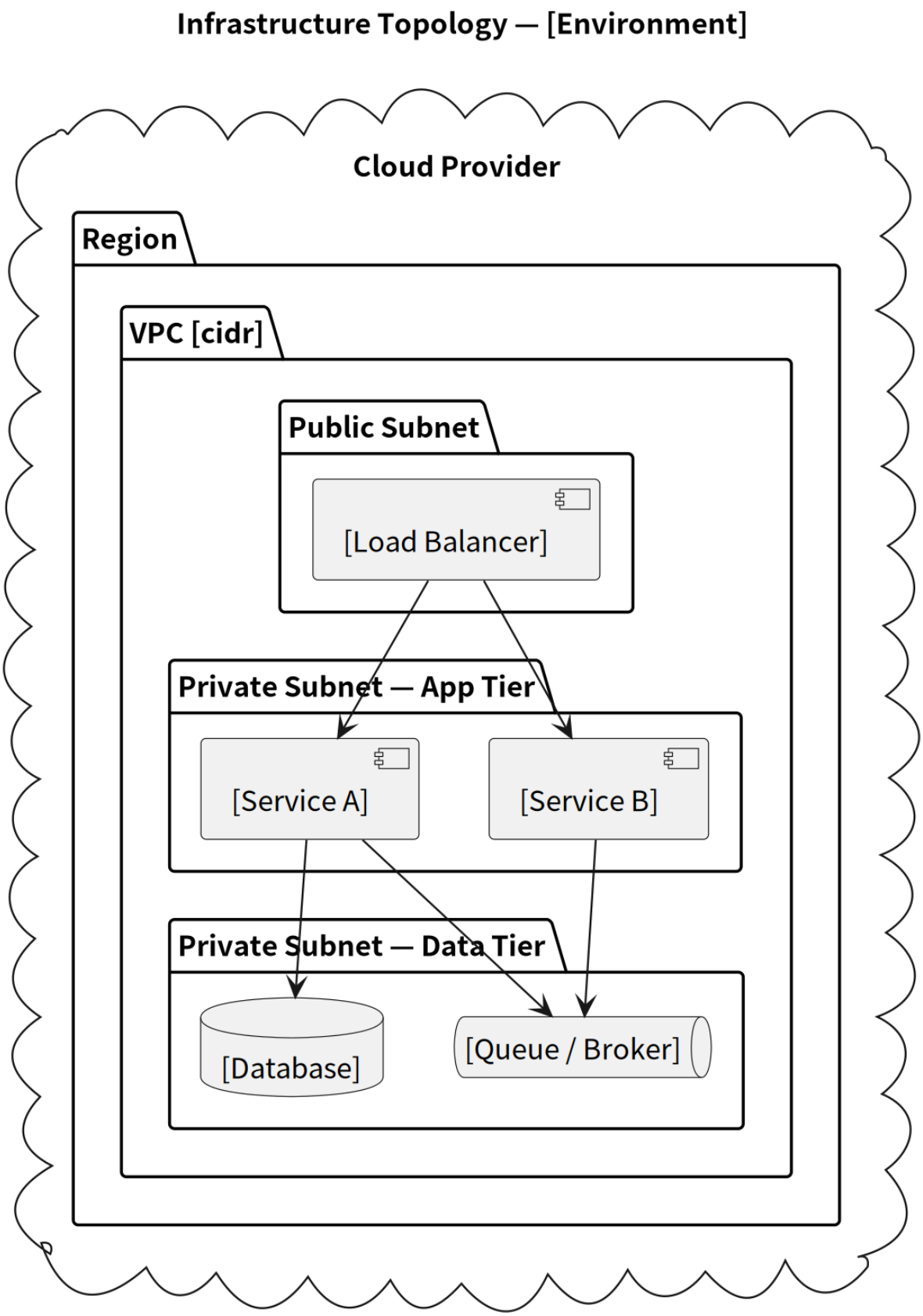
Resource	Type	Environment	Region / Zone	Managed by	Notes
`[resource-name]`	[VPC / Subnet / Instance / DB / LB / Queue / …]	[dev / staging / prod]	[region]	[Terraform module path]	[Notes]

Environment Promotion Path

```
dev  →  staging  →  prod
```

Stage	Purpose	Promotion trigger	Approval required
dev	Active development, ephemeral resources	On every merge to `main`	No
staging	Pre-production validation	Manual promotion after QA sign-off	No
prod	Live traffic	Manual promotion after staging sign-off	Yes — [team/role]

Resource Graph



Secrets and Credential Sources

Secret	Source	Rotation cadence	Owner
`[SECRET_NAME]`	[Vault / SSM / Secrets Manager / env var]	[period or manual]	[team]

Known Constraints

- [e.g. Cross-region latency: service A and DB must be in the same region]
- [e.g. Prod DB is single-AZ — failover is manual]

Data Model

[Entity / Collection / Node Name] (`[table_name / collection_name]`)

Purpose: [What this entity stores]

Field	Type	Constraint	Description
`id`	[e.g., UUID / ObjectId / string]	PK, NOT NULL	Primary key
`[field]`	[type]	[constraint]	[Description]
`[fk_field]_id`	[type]	FK → `[table].id`, NOT NULL	[Description]
`status`	[e.g., ENUM / string]	NOT NULL	See state machine below
`created_at`	[e.g., TIMESTAMPTZ / Date / number]	NOT NULL	Creation timestamp
`updated_at`	[e.g., TIMESTAMPTZ / Date / number]	NOT NULL	Last updated timestamp
`deleted_at`	[e.g., TIMESTAMPTZ / Date / number]	nullable	Soft delete — omit if not used

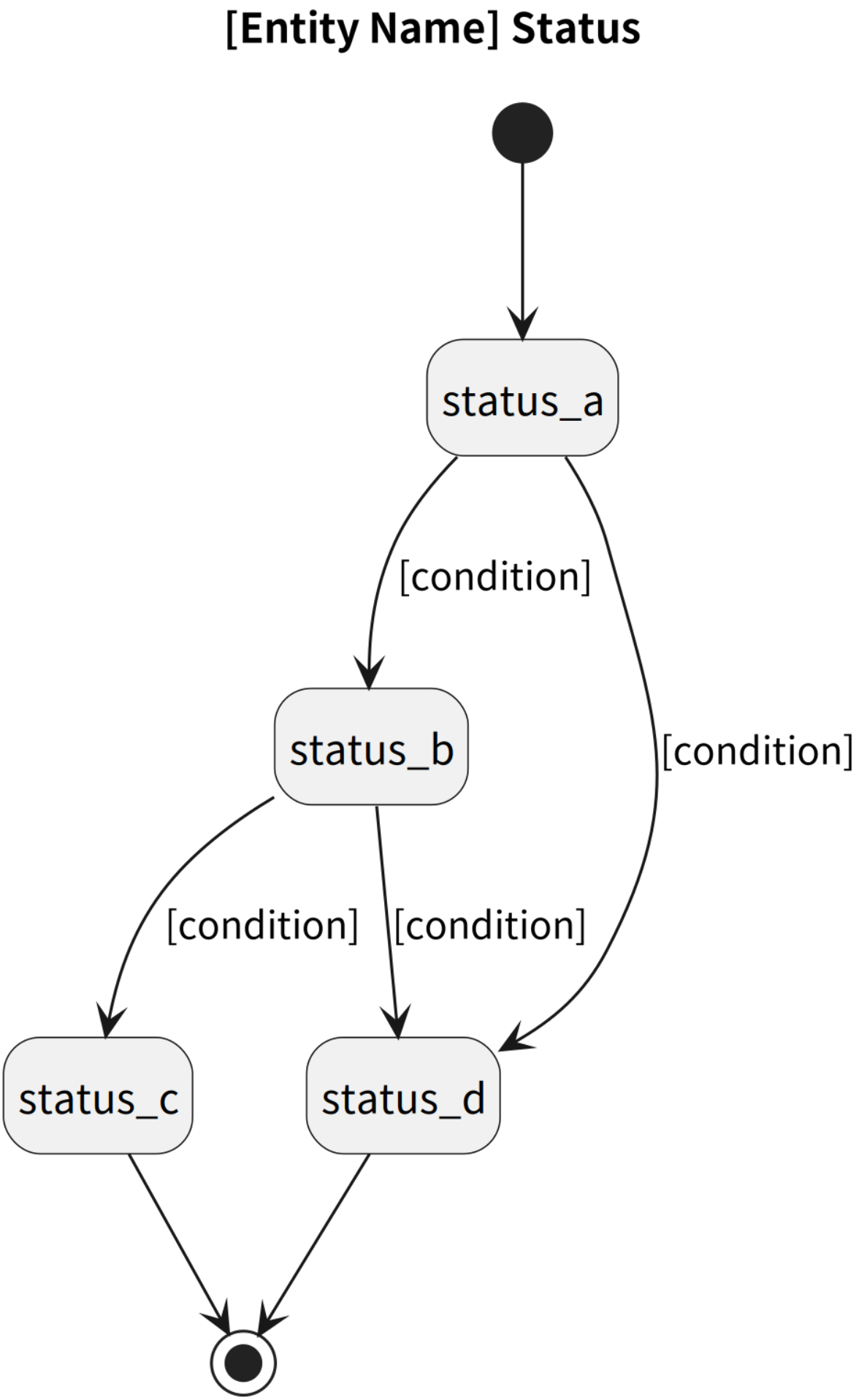
Indexes:

Index name	Field(s)	Purpose
`idx_[table]_[field]`	`[field]`	[Which query this serves]

State Machine:

```
[status_a] → [status_b] → [status_c]
                  ↓
              [status_d]
```

Status	Can transition to
`[status_a]`	`[status_b]`, `[status_d]`
`[status_b]`	`[status_c]`, `[status_d]`
`[status_c]`	—
`[status_d]`	—


[Entity Name] Status
status_a
[condition]
status_b
[condition]
[condition]
[condition]
status_c
status_d

[Entity / Collection / Node Name] (`[table_name / collection_name]`)

Purpose: [Description]

Field	Type	Constraint	Description
`id`	[type]	PK, NOT NULL	Primary key
`[field]`	[type]	[constraint]	[Description]
`created_at`	[type]	NOT NULL	
`updated_at`	[type]	NOT NULL	

Indexes:

Index name	Field(s)	Purpose
`idx_[table]_[field]`	`[field]`	[Which query this serves]

Migration Plan

Order	File / Step	Operation	Reversible
1	`[timestamp]_create_[table]`	CREATE TABLE / create collection	
2	`[timestamp]_add_index_[table]`	CREATE INDEX	
3	`[timestamp]_modify_[col]_[table]`	ALTER TABLE / backfill field	Verify data first

Query Patterns

Query	Condition	Index used
[e.g., List by owner]	`owner_id = ?` AND `deleted_at IS NULL`	`idx_[table]_owner_id`
[e.g., Filter by status]	`status = ?`	`idx_[table]_status`

API Contract

> **Applies to:** Web App, Microservices. For other project types see the routing table above.

Base URL: `[e.g., /api/v1]`
Authentication: `[e.g., Bearer Token (JWT) / API Key / mTLS / None]`
Content-Type: `application/json`

Overview

Method	Path	Description	Auth
`POST`	`/[resource]`	[description]	
`GET`	`/[resource]`	[description]	
`GET`	`/[resource]/:id`	[description]	
`PATCH`	`/[resource]/:id`	[description]	
`DELETE`	`/[resource]/:id`	[description]	

`POST /[resource]`

Description: [description]

Request Body:

```
{
  "[field]": "string",   // required
  "[field]": 0,          // required
  "[field]": "string"    // optional
}
```

Validation Rules:

Field	Required	Rule	Error code
`[field]`		Length 1–255	`VALIDATION_FIELD_REQUIRED`
`[field]`		Range 1–1000	`VALIDATION_FIELD_OUT_OF_RANGE`
`[field]`		Format: email	`VALIDATION_FIELD_FORMAT`

Success `201 Created`:

```
{
  "id": "uuid",
  "[field]": "string",
  "created_at": "2025-01-01T00:00:00Z"
}
```

Errors:

HTTP	Error code	Scenario
`400`	`VALIDATION_FIELD_REQUIRED`	Required field missing

HTTP	Error code	Scenario
`401`	`AUTH_TOKEN_EXPIRED`	Token expired
`409`	`[ENTITY]_ALREADY_EXISTS`	Duplicate resource

`GET /[resource]`

Description: [description]

Query Parameters:

Parameter	Required	Default	Description
`page`		`1`	Page number
`per_page`		`20`	Items per page, max 100
`status`		—	Filter by status

Success `200 OK`:

```
{
  "data": [
    {
      "id": "uuid",
      "[field]": "string",
      "created_at": "2025-01-01T00:00:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "per_page": 20,
    "total": 100,
    "total_pages": 5
  }
}
```

Errors:

HTTP	Error code	Scenario
`400`	`VALIDATION_ENUM_INVALID`	Invalid status value
`401`	`AUTH_TOKEN_EXPIRED`	Token expired

`GET /[resource]/:id`

Description: [description]

Success `200 OK` : Returns the full resource

Errors:

HTTP	Error code	Scenario
`401`	`AUTH_TOKEN_EXPIRED`	Token expired
`403`	`AUTH_RESOURCE_NOT_OWNED`	Accessing another user's resource
`404`	`[ENTITY]_NOT_FOUND`	ID does not exist

`PATCH /[resource]/:id`

Description: Partial update — only send fields to change

Request Body: Same as POST, but all fields are optional

Success `200 OK` : Returns the updated full resource

Errors:

HTTP	Error code	Scenario
`401`	`AUTH_TOKEN_EXPIRED`	Token expired
`403`	`AUTH_RESOURCE_NOT_OWNED`	Modifying another user's resource
`404`	`[ENTITY]_NOT_FOUND`	ID does not exist
`409`	`[ENTITY]_STATE_INVALID`	Current state does not allow this operation
`409`	`[ENTITY]_CONCURRENT_UPDATE`	Optimistic lock conflict

`DELETE /[resource]/:id`

Description: [Soft delete / Hard delete]

Success `204 No Content`

Errors:

HTTP	Error code	Scenario
`401`	`AUTH_TOKEN_EXPIRED`	Token expired
`403`	`AUTH_RESOURCE_NOT_OWNED`	Deleting another user's resource
`404`	`[ENTITY]_NOT_FOUND`	ID does not exist
`409`	`[ENTITY]_STATE_INVALID`	Current state does not allow deletion

Error Response Format

All errors use a unified format:

```
{
  "error": {
    "code": "VALIDATION_FIELD_REQUIRED",
    "message": "Field [field] is required",
    "user_message": "[Field name] is required"
  }
}
```

Error Code Catalogue

Code	HTTP	Description	Retryable
`AUTH_TOKEN_MISSING`	401	Authorization header missing	N
`AUTH_TOKEN_EXPIRED`	401	Token has expired	N
`AUTH_TOKEN_INVALID`	401	Token verification failed	N
`AUTH_PERMISSION_DENIED`	403	Role lacks required permission	N
`AUTH_RESOURCE_NOT_OWNED`	403	Resource does not belong to current user	N
`VALIDATION_FIELD_REQUIRED`	400	Required field missing	Y
`VALIDATION_FIELD_FORMAT`	400	Field format invalid	Y
`VALIDATION_FIELD_TOO_LONG`	400	Field exceeds maximum length	Y
`VALIDATION_FIELD_OUT_OF_RANGE`	400	Value out of allowed range	Y
`VALIDATION_ENUM_INVALID`	400	Invalid enum value	Y
`[ENTITY]_NOT_FOUND`	404	Resource does not exist	N
`[ENTITY]_DELETED`	410	Resource has been permanently deleted	N
`[ENTITY]_ALREADY_EXISTS`	409	Resource already exists	N
`[ENTITY]_STATE_INVALID`	409	Current state does not allow this operation	N
`[ENTITY]_CONCURRENT_UPDATE`	409	Optimistic lock conflict	Y
`RATE_LIMIT_EXCEEDED`	429	Too many requests	Y
`EXTERNAL_[SERVICE]_TIMEOUT`	504	Upstream service timed out	Y
`INTERNAL_UNEXPECTED`	500	Unexpected internal error	Y

WebSocket Events

Transport: `[e.g., Socket.IO 4.x / native WebSocket]`
Connection URL: `[e.g., wss://api.example.com / /socket.io]`
Authentication: `[e.g., JWT passed as auth.token in handshake / cookie / query param]`

Event Envelope

```
{
  "id":        "uuid",
  "type":      "event:name",
  "createdAt": "2025-01-01T00:00:00Z",
  "payload":   {}
}
```

Event Overview

Event	Direction	Trigger	Payload type
`[event:name]`	server → client	[what triggers it]	[payload type or —]
`[event:name]`	client → server	[what the client sends]	[payload type or —]

Server → Client Events

`[event:name]`

Trigger: [what causes the server to emit this event]

Payload:

```
{
  "[field]": "[type / example value]"
}
```

Client → Server Events

`[event:name]`

Description: [what this event does]

Payload:

```
{
  "[field]": "[type / example value]"
}
```

Response / Acknowledgement: [describe ack if any, or "none"]

Connection Lifecycle

Event	Direction	Description
`connect`	client → server	Client establishes connection
`disconnect`	server → client	Server closes connection
`[custom]`	[direction]	[description]

Error Handling

Scenario	Behaviour
Authentication failure	[e.g., connection refused with 401]
Invalid payload	[e.g., server emits error event with code]
Connection drop	[e.g., client reconnects automatically]

GraphQL API

Endpoint: `[e.g., POST /graphql]`
Authentication: `[e.g., Authorization: Bearer <token> header]`

Types

```
type [TypeName] {
  [field]: [Type]
}

input [InputName] {
  [field]: [Type]  # required / optional
}
```

Queries

Query	Description	Auth
`[queryName]([args])`	[description]	/

`[queryName]`

Arguments:

Argument	Type	Required	Description
`[arg]`	`[Type]`		[description]

Returns: `[TypeName]`

Errors:

Code	Scenario
`[ERROR_CODE]`	[scenario]

Mutations

Mutation	Description	Auth
`[mutationName]([args])`	[description]	/

`[mutationName]`

Arguments:

Argument	Type	Required	Description
`[arg]`	`[Type]`		[description]

Returns: `[TypeName]`

Errors:

Code	Scenario
`[ERROR_CODE]`	[scenario]

Subscriptions

Subscription	Description	Trigger
`[subscriptionName]`	[description]	[what triggers it]

---

gRPC API

Proto package: `[e.g., com.example.v1]`
Server address: `[e.g., grpc.example.com:443]`
Authentication: `[e.g., JWT in Authorization metadata / mTLS]`

Service Overview

RPC Method	Type	Description
`[ServiceName]/[MethodName]`	Unary / Server streaming / Client streaming / Bidirectional	[description]

`[ServiceName]`

`[MethodName]`

Type: `[Unary / Server streaming / Client streaming / Bidirectional]`

Request: `[MessageType]`

Field	Type	Required	Description
`[field]`	`[proto type]`		[description]

Response: `[MessageType]`

Field	Type	Description
`[field]`	`[proto type]`	[description]

Errors (gRPC status codes):

Status code	Scenario
`NOT_FOUND`	[scenario]
`INVALID_ARGUMENT`	[scenario]
`PERMISSION_DENIED`	[scenario]

CLI Commands

Binary: `[e.g., myapp / npx myapp / python -m myapp]`

Command Overview

Command	Description
`[binary] [command]`	[description]
`[binary] [command] [subcommand]`	[description]

`[binary] [command]`

Description: [what this command does]

Usage:

```
[binary] [command] [flags] [arguments]
```

Flags:

Flag	Short	Required	Default	Description
`--[flag]`	`-[f]`		—	[description]
`--[flag]`	—		`[default]`	[description]

Arguments:

Argument	Required	Description
`[arg]`		[description]

Output:

```
[example output format]
```

Exit codes:

Code	Meaning
`0`	Success
`1`	[error scenario]

Examples:

```
## Permissions

**Access Control Model:** [RBAC / ABAC / ACL / Ownership-only / other]

---

## Role Definitions

| Role | Description | Inherits from |
|---|---|---|
| `[ROLE_NAME]` | [Who this role represents and what they can access] | [— or
parent role] |
| `[ROLE_NAME]` | [Description] | [— or parent role] |

---
```

```
## Permission Definitions

| Permission | Description |
|---|---|
| `[resource]:read` | Read [resource] |
| `[resource]:create` | Create [resource] |
| `[resource]:update` | Update own [resource] |
| `[resource]:update:any` | Update any [resource] |
| `[resource]:delete` | Delete own [resource] |
| `[resource]:delete:any` | Delete any [resource] |

---

## RBAC Matrix

| Permission | ROLE_GUEST | ROLE_USER | ROLE_ADMIN |
|---|---|---|---|
| `[resource]:read` |  |  |  |
| `[resource]:create` |  |  |  |
| `[resource]:update` |  |  |  |
| `[resource]:update:any` |  |  |  |
| `[resource]:delete` |  |  |  |
| `[resource]:delete:any` |  |  |  |

**   Conditions:**
* `[resource]:update` — `ROLE_USER` may only update resources where `owner_id = current_user.id`
* `[resource]:delete` — `ROLE_USER` may only delete resources where `owner_id = current_user.id`

---

## API Endpoint Access

| Method | Path | Required permission | Minimum role | Source | Extra condition |
|---|---|---|---|---|---|
| `POST` | `/[resource]` | `[resource]:create` | `ROLE_USER` | [Hardcoded / Seeded default] | — |
| `GET` | `/[resource]` | `[resource]:read` | `ROLE_GUEST` | [Hardcoded / Seeded default] | — |
| `GET` | `/[resource]/:id` | `[resource]:read` | `ROLE_GUEST` | [Hardcoded / Seeded default] | — |
| `PATCH` | `/[resource]/:id` | `[resource]:update` | `ROLE_USER` | [Hardcoded / Seeded default] | Own resource only |
| `DELETE` | `/[resource]/:id` | `[resource]:delete` | `ROLE_USER` | [Hardcoded / Seeded default] | Own resource only |

---
```

```
## Enforcement Layers

| Layer | Responsibility | Applies to |
|---|---|---|
| API Gateway | Token validation, role extraction | Web App / Microservices |
| Middleware / Route Guard | Role-permission check, reject unauthorized | Web App / Microservices |
| Service Layer | Ownership check — `owner_id = current_user.id` | Web App |
| CLI Flag / Config | Require elevated flag or config key for privileged commands | CLI Tool |
| API Key Header | Validate key before processing any request | AI/LLM App / Library |
| Pipeline Trigger Auth | Verify caller identity before allowing pipeline run | Data Pipeline |

---

## Edge Cases

| Edge Case | Design | Response |
|---|---|---|
| Unauthenticated access to protected resource | Token / key check at entry point fails | Reject with auth error (HTTP 401 / non-zero exit / exception) |
| Low-privilege role attempts high-privilege action | Role-permission check | Reject (HTTP 403 / permission denied error) |
| User accesses another user's resource | Ownership check in service layer | Reject (HTTP 403 / access denied) |
| Expired / revoked token or API key | Token validation fails | Reject with auth error |
| Privileged CLI command run without required flag | Arg parser check | Exit with usage error |
| Pipeline triggered by unauthorized caller | Trigger auth check | Reject run request |

---

## Use Case Diagram

![](.pdf_build_cache/permissions.png)



## Pipeline Contract


## Overview

| Stage | Input | Output | Trigger |
|---|---|---|---|
```

```
---

## Stage Contracts

Repeat this block for each pipeline stage.

---

## Cross-Stage Consistency Check

Run this check whenever any stage contract changes.

For each field or file that crosses a stage boundary:

| Field / File | Produced by | Consumed by | Format match? | Verified |
|---|---|---|---|---|
| [field name or file path] | [stage] | [stage] |    /    | [date] |

A mismatch between produced and consumed format is a **High** finding.
Do not proceed to the next stage's implementation until the mismatch is resolved.


## CLI Contract


## Command Structure
```

[tool-name] [flags] [arguments]

```
Global flags (apply to all subcommands):

| Flag | Type | Default | Description |
|---|---|---|---|
| `--help`, `-h` | bool | false | Show help |
| `--version`, `-v` | bool | false | Show version |
| `--config` | string | `~/.toolname.yaml` | Path to config file |

---

## Subcommands

Repeat this block for each subcommand.

---
```

```
### `[tool-name] [subcommand]`

**Description:** [One line — what this command does]

**Usage:**
```

[tool-name] [subcommand] [flags] [optional-arg]

```
#### Arguments

| Name | Required | Description |
|---|---|---|
| `<required-arg>` | Yes | [What it is] |
| `[optional-arg]` | No | [What it is, default behaviour if omitted] |

#### Flags

| Flag | Short | Type | Default | Description |
|---|---|---|---|---|
| `--output` | `-o` | string | stdout | Output file path |
| `--format` | `-f` | enum (json\|yaml\|table) | table | Output format |
| `--dry-run` | | bool | false | Preview without writing |

#### Output

**Success (exit 0):**
```

[Example stdout output — use real representative output]

```
**Failure (exit 1):**
```

Error: [example error message]

```
#### Exit Codes

| Code | Meaning |
|---|---|
| 0 | Success |
| 1 | General error (see stderr for details) |
| 2 | Invalid arguments or flags |

#### Example

```bash
# [tool-name] config (~/.toolname.yaml or .toolname.yaml in project root)
```
```

```
[key]: [value]
[section]:
  [nested-key]: [value]
```

Config file values are overridden by flags. Flags always take precedence.

Environment Variables

Variable	Default	Description
`TOOLNAME_CONFIG`	`~/.toolname.yaml`	Config file path override
`TOOLNAME_LOG_LEVEL`	`info`	Log verbosity (debug / info / warn / error)

Stdin / Stdout / Stderr Contract

Stream	Content
stdin	[Accepted if `--input -` is passed / Not used]
stdout	[Machine-readable output — JSON / table / plain text]
stderr	[Human-readable progress and error messages only]

Rule: stdout must be safe to pipe. Never mix progress messages into stdout.

Public API

Public API Summary

Package name: `[package-name]`
Import:

```
# Python
from package_name import ClassName, function_name
```

```
// TypeScript / JavaScript
import { ClassName, functionName } from 'package-name'
```

Stability tiers

Tier	Meaning
Stable	Will not change without a major version bump

Tier	Meaning
Beta	May change in minor versions — caller must pin
Internal	Not part of public API — do not use directly

Functions

Repeat this block for each public function.

`functionName(param1, param2)`

Stability: Stable / Beta

Description: [What it does — one to three sentences]

Parameters:

Name	Type	Required	Description
`param1`	`string`	Yes	[Description]
`param2`	`int`	No	[Description. Default: `0`]

Returns: `ReturnType` — [Description of what is returned]

Raises / Throws:

Exception	When
`ValueError`	[Condition that triggers this error]
`ConnectionError`	[Condition that triggers this error]

Example:

```
result = function_name("input", param2=5)
```

Classes

Repeat this block for each public class.

`ClassName`

Stability: Stable / Beta

Description: [What this class represents or does]

Constructor:

```
obj = ClassName(required_param, optional_param=default)
```

Parameter	Type	Required	Description
`required_param`	`string`	Yes	[Description]
`optional_param`	`bool`	No	[Description. Default: `False`]

Public methods:

Method	Returns	Description
`method_name(arg)`	`ReturnType`	[What it does]
`other_method()`	`None`	[What it does]

Example:

```
obj = ClassName("value")
result = obj.method_name("arg")
```

Types and Enums

```
class StatusEnum(Enum):
    PENDING = "pending"
    ACTIVE  = "active"
    DONE    = "done"
```

Constants

Name	Value	Description
`DEFAULT_TIMEOUT`	`30`	Default request timeout in seconds
`MAX_RETRIES`	`3`	Default retry limit

What is NOT public

The following are internal implementation details and must not be imported or relied on directly. They may change in any version without notice.

- `_internal_module`
- `_PrivateClass`

- Any symbol beginning with `_`

Deprecation Log

Symbol	Deprecated in	Removed in	Migration
`old_function()`	v1.2.0	v2.0.0	Use `new_function()` instead

Model Contract

Model Identity

Property	Value
Model name	[e.g., `revenue-forecast-v1`]
Type	[Classification / Regression / Ranking / Generation / etc.]
Purpose	[One sentence — what business problem it solves]
Framework	[scikit-learn / PyTorch / TensorFlow / XGBoost / etc.]
Artifact registry	[MLflow / S3 path / HuggingFace Hub / etc.]

Input Contract

Feature Schema

Feature name	Type	Required	Range / Categories	Description
`feature_1`	float	Yes	[0, 1]	[Description]
`feature_2`	int	Yes	{1, 2, 3, 4}	[Description]
`feature_3`	string	No	max 255 chars	[Description. Null → treated as "unknown"]

Preprocessing expectations

[Describe what preprocessing the serving layer must apply before passing data to the model]

- Normalization: [which features, which scaler — must use the same scaler fitted during training]
- Encoding: [which features, which encoder]
- Missing value strategy: [imputation method per feature]

Note: If the serving layer uses a different preprocessing than training, predictions will be wrong. Document the exact fitted transformer artifact location.

Output Contract

Field	Type	Description
`prediction`	float / int / string	[Primary model output]
`confidence`	float [0, 1]	[Confidence score — may be absent for some model types]
`label`	string	[Decoded class label — if applicable]

Output format: [Single value / Dict / JSON / Pandas DataFrame row]

Example output:

```
{
  "prediction": 0.87,
  "confidence": 0.92,
  "label": "high_risk"
}
```

Production Thresholds

A model version must meet ALL thresholds below before being tagged `production`.

Metric	Threshold	Evaluation dataset
Accuracy / RMSE / AUC	>= [value]	[dataset name and version]
Precision	>= [value]	—
Recall	>= [value]	—
Latency (p99)	<= [ms]	[load test scenario]

Threshold rationale: [Why these thresholds — business impact of a miss]

Known Limitations

Limitation	Impact	Mitigation

Training Data Requirements

Property	Value
Dataset name	[Name and version / DVC tag / S3 path]

Property	Value
Date range	[e.g., 2022-01-01 to 2024-12-31]
Minimum rows for training	[N]
Label source	[How ground truth was obtained]
Known data quality issues	[Any known biases, missing periods, or anomalies]

Retraining Policy

Trigger	Action
Model AUC drops below [threshold] on production traffic	Trigger retraining pipeline
New training data available (monthly / quarterly)	Scheduled retraining
Feature schema change	Mandatory retraining — old model is incompatible

Service Catalog

Services

Service	Type	Owner	Port	Base URL	Status
[service-name]	[API / Worker / Gateway / Frontend]	[team or person]	[port]	[http://service-name:port]	Active / Deprecated

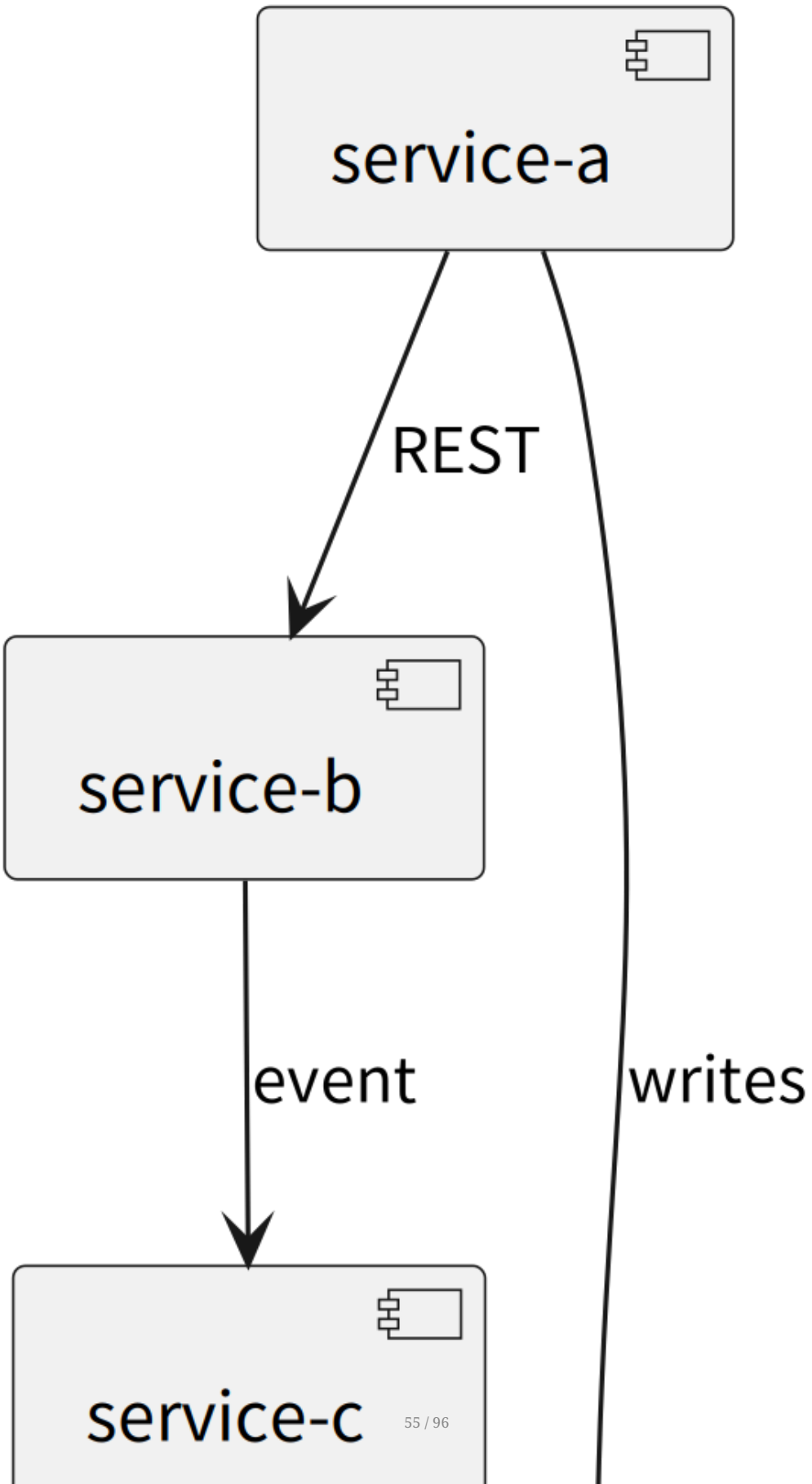
---

Service Details

Repeat this block for each service.

Dependency Graph

Service Dependency Graph



Deprecated Services

Service	Deprecated	Replacement	Shutdown date
[old-service]	YYYY-MM-DD	[new-service]	YYYY-MM-DD

Service Contract

REST / HTTP Contracts

Repeat this block for each service-to-service REST call.

Event / Message Contracts

Repeat this block for each event that crosses service boundaries.

Schema Evolution Rules

1. **Additive changes** (new optional field): consumers must ignore unknown fields — safe to deploy without coordination.
2. **Breaking changes** (rename, remove, type change): bump `version`, keep old consumer alive until all consumers are migrated, then remove old version.
3. **Never** remove a field without a deprecation period of at least one release cycle.

LLM Contract

Model Configuration

Property	Value
Provider	[e.g., Anthropic, OpenAI, local Ollama]
Model ID	[e.g., claude-opus-4-7, gpt-4o, llama3.2]
API endpoint	[URL or SDK reference]
Context window	[tokens — e.g., 200k]
Max output tokens	[tokens]

Parameters

Parameter	Value	Rationale
temperature	[0.0–2.0]	[why — e.g., 0.3 for factual, 0.8 for creative]
top_p	[0.0–1.0]	[note if not used]
max_tokens	[tokens]	[upper bound per response]
stop sequences	[list or "none"]	

System Prompt

Version: `[v1 / date / git tag]`

> **If the system prompt exceeds ~400 tokens:** store it in a separate file (`docs/specs/system-prompt-[version].md`) and reference it here by path. Paste only the current version inline when it fits within one screen.

```
[Paste the full system prompt here.
Include persona, domain constraints, tone, and any output format instructions.]
```

Key design decisions in this prompt:

Decision	Rationale

Tool Calling / Function Schemas

[List each tool the model can call. Skip this section if no tools are used.]

Tool: `[tool_name]`

```
{
  "name": "[tool_name]",
  "description": "[what this tool does and when the model should call it]",
  "input_schema": {
    "type": "object",
    "properties": {
      "[param]": { "type": "[string|number|boolean]", "description": "[what it
means]" }
    },
    "required": ["[param]"]
```

```
  }
}
```

Context Window Strategy

How conversation history is managed when context grows long:

Strategy	Detail
History trimming	[e.g., keep last N turns / summarize oldest turns / sliding window]
System prompt position	[always first / re-injected every N turns]
RAG injection point	[before user message / after system prompt — if applicable]

Streaming

Property	Value
Streaming enabled	[Yes / No]
Partial render	[Yes — UI updates as tokens arrive / No — wait for full response]

Retry and Fallback

Scenario	Behaviour
API timeout / 5xx	[retry N times with exponential backoff]
Rate limit (429)	[wait and retry / surface error to user]
Content filter block	[surface refusal message / fallback response]
Model unavailable	[fallback to: [model ID] / fail with error]

prompt-library

Active Prompts

Prompt ID	Current version	Purpose	File
[prompt-id]	v1	[what it does]	`prompts/[prompt-id]-prompt.md`

RAG Contract

Knowledge Sources

Source	Type	Update frequency	Format

Chunking Strategy

Property	Value	Rationale
Chunk size	[e.g., 512 tokens]	[why — e.g., balances context vs precision]
Overlap	[e.g., 64 tokens]	[prevents context loss at boundaries]
Splitter	[e.g., RecursiveCharacterTextSplitter / sentence splitter]	
Metadata attached	[e.g., source name, date, section title]	[used for filtering and citation]

Embedding Model

Property	Value
Model	[e.g., text-embedding-3-small, voyage-finance-2]
Provider	[OpenAI / Anthropic / local]
Dimensions	[e.g., 1536]
Re-embedding trigger	[e.g., on source update / when model changes]

Vector Store

Property	Value
Store	[e.g., Chroma, Pinecone, pgvector, FAISS]
Index type	[e.g., HNSW, IVFFlat]
Similarity metric	[cosine / dot product / L2]
Similarity threshold	[e.g., 0.75 — below this, result is excluded]
Top-K retrieved	[e.g., 5 chunks per query]

Retrieval Flow

Context Injection Format

How retrieved chunks are formatted before injecting into the prompt:

```
[CONTEXT]
Source: {{source_name}} ({{date}})
{{chunk_text}}

Source: {{source_name}} ({{date}})
{{chunk_text}}
[/CONTEXT]
```

Failure Handling

Scenario	Behaviour
No chunks above threshold	[Proceed without context / inform user retrieval failed / refuse to answer]
Vector store unavailable	[Fallback: answer from model knowledge only / surface error]
Stale index (source updated but not re-embedded)	[Log warning / flag in response]

MCP Contract

MCP Client Configuration

Property	Value
MCP SDK / client library	[e.g., @anthropic-ai/mcp, mcp (Python), Claude Desktop built-in]
Config file location	[e.g., claude_desktop_config.json / mcp_config.json / inline in app code]
Default transport	[stdio / SSE / HTTP Streaming]

Connected Servers

Server name	Transport	Purpose	Status
`[server-name]`	[stdio / SSE / HTTP]	[What this server provides]	[Active / Disabled]

Server name	Transport	Purpose	Status
`[server-name]`	[stdio / SSE / HTTP]	[What this server provides]	[Active / Disabled]

Server Detail

`[server-name]`

Transport: `stdio`
Command: `[e.g., npx / uvx / python -m]`
Args: `["[package-or-path]", "[flag]", "[value]"]`

Auth: `[None / API key header X-API-Key / OAuth / mTLS]`
Version / package: `[e.g., @modelcontextprotocol/server-filesystem@0.6.2]`

Tools

Tool name	Description	When LLM should call it
`[tool_name]`	[What it does]	[Trigger condition — e.g., "user asks about current prices"]

Tool schema — `[tool_name]`:

```
{
  "name": "[tool_name]",
  "description": "[what this tool does and when to call it]",
  "inputSchema": {
    "type": "object",
    "properties": {
      "[param]": {
        "type": "[string | number | boolean | array | object]",
        "description": "[what this parameter means]"
      }
    },
    "required": ["[param]"]
  }
}
```

Expected output format:

```
{
  "content": [
    { "type": "text", "text": "[example output]" }
  ]
}
```

Resources

URI template	MIME type	Description
`[e.g., file:///data/{filename}]`	`[e.g., text/plain]`	[What this resource contains]
`[e.g., resource://[server]/[name]]`	`[MIME type]`	[Description]

Prompt Templates

Template name	Arguments	Description
`[template-name]`	`[arg1, arg2]`	[What this template generates]

Tool Use Policy

Property	Value
Max tool calls per turn	[e.g., 5 / unlimited]
Parallel tool calls	[Allowed / Disabled]
Tool call confirmation	[Silent — execute immediately / Prompt user before executing]
Restricted tools	[e.g., none / list any tools the LLM must not call autonomously]

System prompt hints for tool use:

> Add these instructions to the system prompt in `llm-contract.md` so the LLM knows when and how to use each tool.

```
[e.g.:
- Use the `search_web` tool whenever the user asks about current events or prices.
- Always call `get_portfolio` before giving any personalised advice.
- Do not call `execute_trade` without explicit user confirmation in the same turn.
]
```

Failure Handling

Scenario	Behaviour
Tool call timeout	[e.g., abort after 10 s, return error text to LLM, ask user to retry]
Tool returns error	[e.g., pass error message back to LLM as tool result, let LLM explain to user]
Server process crashes (stdio)	[e.g., restart server process, surface reconnection error]

Scenario	Behaviour
Server unreachable (SSE / HTTP)	[e.g., disable affected tools, notify user, continue with remaining servers]
Schema mismatch (LLM sends wrong args)	[e.g., server returns validation error → LLM self-corrects on next turn]

Mobile Contract

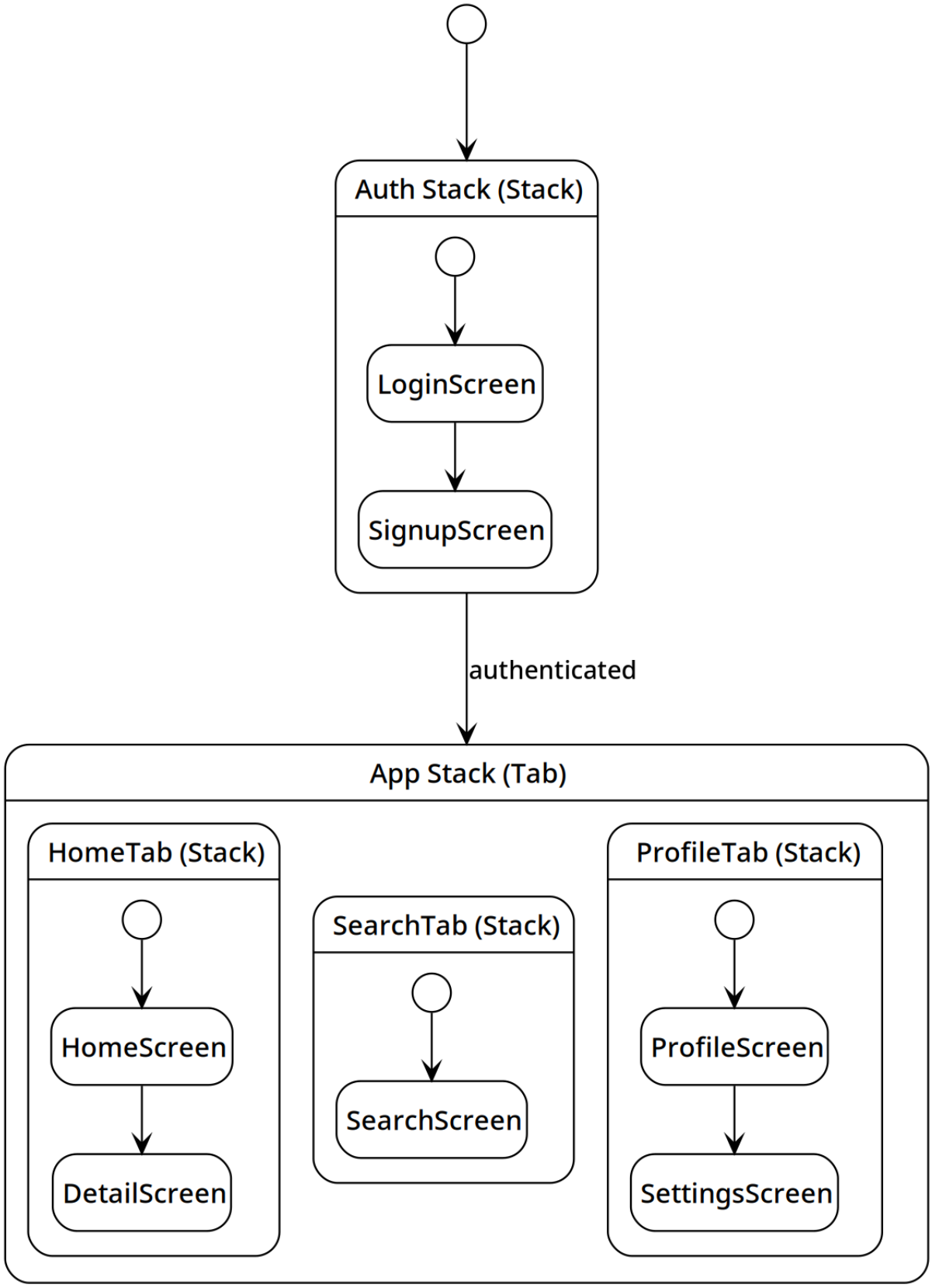
App Identity

Property	Value
App name (display)	[e.g., MyApp]
Bundle ID / App ID	[e.g., com.example.myapp]
Framework	[React Native / Flutter / SwiftUI / Jetpack Compose]
Platforms	[iOS / Android / both]
Min OS version	iOS [X.X] / Android API [XX]

Screen Inventory

Screen name	Route / path	Auth required	Description
[HomeScreen]	`/home`	Yes	[Main dashboard]
[LoginScreen]	`/login`	No	[Credential entry]
[ProfileScreen]	`/profile`	Yes	[User profile view and edit]
[SettingsScreen]	`/settings`	Yes	[App preferences]

Navigation Graph



Deep Link Scheme

Deep link URL	Target screen	Parameters	Notes
`myapp://home`	HomeScreen	—	Opens main dashboard
`myapp://item/:id`	DetailScreen	`id: string`	Opens specific item
`https://myapp.com/share/:id`	ShareScreen	`id: string`	Universal link for sharing

Scheme name: `myapp://`
Universal link domain: `[myapp.com]` (if configured)

OS Permission Declarations

Permission	Platform	When requested	Required / Optional	Reason shown to user
Camera	iOS + Android	On first photo upload	Optional	"To let you upload profile photos"
Push Notifications	iOS + Android	After first login	Optional	"To send you order updates"
Location (when in use)	iOS + Android	On map feature first use	Optional	"To show nearby results"
Photo Library	iOS	On profile photo upload	Optional	"To select a photo from your library"

iOS: declare all permissions in `Info.plist` with usage description strings.
Android: declare all permissions in `AndroidManifest.xml`.

Push Notification Payloads

[notification-type-1] Notification

Trigger: [e.g., order status update from backend]

```
{
  "notification": {
    "title": "Your order is on the way",
    "body": "Order #1234 has shipped"
  },
  "data": {
    "type": "order_update",
    "order_id": "1234",
    "status": "shipped",
```

```
    "deep_link": "myapp://orders/1234"
  }
}
```

Handling: Tap opens DetailScreen for `order_id`. Background: update order status badge.

[notification-type-2] Notification

Trigger: [e.g., chat message received]

```
{
  "notification": {
    "title": "[Sender name]",
    "body": "[Message preview — max 50 chars]"
  },
  "data": {
    "type": "chat_message",
    "thread_id": "[thread-id]",
    "deep_link": "myapp://chat/[thread-id]"
  }
}
```

Handling: Tap opens ChatScreen for `thread_id`. Background: increment badge count.

App Store / Play Store Metadata

Field	iOS (App Store)	Android (Google Play)
Category	[Utilities / Social / etc.]	[Tools / Communication / etc.]
Age rating	[4+ / 12+ / 17+]	[Everyone / Teen / Mature]
In-app purchases	[Yes / No]	[Yes / No]
Privacy policy URL	[URL]	[URL]

Platform-Specific Notes

iOS

- **Signing:** Distribution certificate + provisioning profile required for App Store build
- **App Transport Security:** [List any ATS exceptions in Info.plist]
- **Background modes:** [e.g., Background fetch, Remote notifications — if used]

Android

- **Signing:** Keystore required; `upload-keystore.jks` stored in [location, never in git]
- **Target SDK:** API [XX] (must be current Play Store requirement)
- **Proguard / R8:** [Enabled / Disabled]; rules in `proguard-rules.pro`

4. Build

Codebase Map

Page Structure Overview

```
title: Page Structure

component "[e.g., Auth Pages — Login / Register]" as Auth {
  provides: [e.g., login form, register form]
  requires: [e.g., API Client]
}

component "[e.g., Dashboard]" as Dashboard {
  provides: [e.g., overview, stats]
  requires: [e.g., API Client, Auth]
}

component "[e.g., [Feature] Pages]" as Feature {
  provides: [e.g., list view, detail view, form]
  requires: [e.g., API Client, Auth]
}

component "[e.g., API Client]" as API {
  provides: [e.g., HTTP calls, auth headers]
  requires: [e.g., Backend API]
}

Auth --> API : calls
Dashboard --> API : calls
Feature --> API : calls
```

Project Structure

```
[project-root]/
├── docs/                          ← planning, specs, architecture docs
│   ├── architecture/              ← system structure docs
│   ├── business/                  ← business process and object docs
│   ├── modules/                   ← per-module flow and log files
│   ├── specs/                     ← data model, API contract, permissions,
logging
│   └── script/                    ← diagram generators, PDF builder, scan tool
```

```
│
├── [src]/                          ← application source code
│   ├── [module-a]/                 ← [short description, e.g., order management]
│   ├── [module-b]/                 ← [short description]
│   ├── [jobs]/                     ← [background jobs, queue consumers]
│   └── [lib]/              —       ← [shared utilities, no flow file needed]
│
├── [migrations/ or schema/]        ← database schema and migrations (omit for
CLI/Library/LLM script)
├── [docker-compose.yml]            ← local infrastructure services
├── [package.json / go.mod / etc.]  ← dependency manifest
└── ...
```

Coverage Summary

Module / Folder	Type	Status	Flow file
`[src/module-a]`	Feature	Documented	`docs/modules/module-a/module-a-module-data-flow.md`
`[src/jobs/order-consumer]`	Background Job	Documented	`docs/modules/order-consumer/order-consumer-module-data-flow.md`
`[src/module-b]`	Feature	Not documented	—
`[src/lib]`	Shared Utility	— Not required	—

Dependencies

Runtime Dependencies

Package	Version	Purpose

Dev Dependencies

Package	Version	Purpose

External Services

Service	Provider	Used by	Fallback behaviour

Infrastructure

Component	Technology	Version	Purpose

architecture/deployment.md

Purpose:
Describe runtime structure — how to run this system locally and how to deploy it.

Include:
- Services
- Environment variables
- Local startup flow
- Build/deploy flow
- Verification steps

Avoid:
- Business requirements
- Feature details

Services

Service	Technology	Port	Description

Environment Variables

Variable	Required	Description	Example
`[VAR_NAME]`		[what it controls]	`[example value]`
`[VAR_NAME]`		[what it controls, and what the default is]	`[example value]`

Copy `.env.example` to `.env` and fill in the values before starting.

Local Startup Flow

Prerequisites:
- [required tools and versions, e.g., Docker Desktop, Node.js 22+, Python 3.12+]

```
# 1. [Step description]
[command]

# 2. [Step description]
[command]

# 3. [Step description]
[command]
```

Expected state when running:
- [service] available at [URL or description]
- [health check command and expected output]

Build / Deploy Flow

Build

```
[build command(s)]
```

Deploy

```
[deploy command(s)]
```

Environments

Environment	URL	Notes
Local	`http://localhost:[port]`	[local setup description]
Staging	`[URL]`	[hosting platform]
Production	`[URL]`	[hosting platform]

Verification

How to confirm the system is running correctly after startup.

- [ ] **[What to verify]**
  Run: `[exact command]`
  Expected: `[exact output]`

Cache Policy

Key / Layer	TTL	Invalidation trigger	Boundary handling
`[cache key or layer name]`	[e.g., 2s]	[e.g., write to DB, explicit flush]	[e.g., stale reads possible for up to TTL after write; consumer retries on state mismatch]

Boundary conditions:

Scenario	Expected behaviour	Consumer action
Write occurs within TTL window	Cache may serve stale data for up to [TTL]	[e.g., poll again after TTL / show loading state / use optimistic UI]
Cache miss	[e.g., read-through from DB / return 503]	[e.g., retry with backoff]
Invalidation fails	[e.g., stale entry persists until TTL expires]	[e.g., force-refresh button / automatic retry]

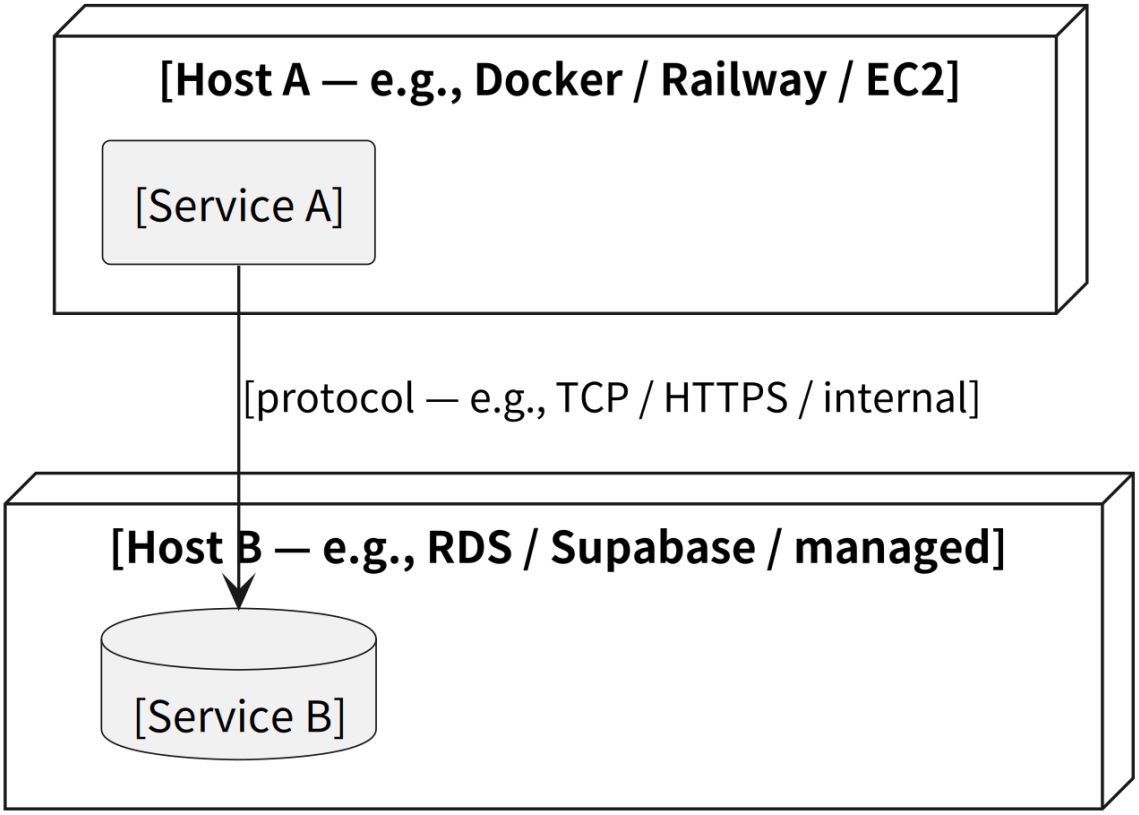
Service-Specific Gotchas

Service	Wrong assumption	Actual behaviour	Verify with

Teardown

```
[command to stop all services]
```

Deployment Diagram



Experiment Log

Log Format

Each entry must include: experiment ID, date, hypothesis, config snapshot, results, and decision.

Do not record an experiment without recording its decision. "No conclusion yet" is a valid decision — it means you will run a follow-up experiment.

Entries

[EXP-000] Baseline

Date: YYYY-MM-DD
Tracking URI: [baseline run reference]

Hypothesis: Establish baseline performance before any optimisation.

Config:

Parameter	Value
Model type	[simplest reasonable model — e.g., LogisticRegression]
Training data	[dataset name + version]
Features	[all raw features, no engineering]
Hyperparameters	defaults
Train / Val split	[80/20 random]

Results:

Metric	Validation

Decision: Baseline established. All future experiments compare against this run.

Evaluation Spec

Judge Model

Property	Value
Judge model	[e.g., claude-opus-4-7]
Judge temperature	[e.g., 0.0 — deterministic scoring]
Scoring format	[1–5 per criterion / pass-fail / weighted average]
Pass threshold	[e.g., average ≥ 3.5 across all criteria]

Eval run results → `docs/specs/eval-log.md`

Evaluation Criteria

Criterion	Weight	1 (Poor)	5 (Excellent)
Factual accuracy	[e.g., 30%]	Contains false or invented facts	All claims are accurate and verifiable
Relevance	[e.g., 25%]	Ignores the user's actual question	Directly addresses what was asked
Risk disclosure	[e.g., 25%]	No mention of risk or uncertainty	Appropriate caveats and risk language present
Clarity	[e.g., 20%]	Confusing, hard to follow	Clear, structured, easy to act on

Add or remove criteria to match your application's quality goals.

Judge Prompt Template

```
You are evaluating the quality of a [financial advisor / assistant / etc.] AI
response.

User question:
{{user_question}}

AI response to evaluate:
{{ai_response}}

Score the response on each criterion below. For each criterion, give:
- A score from 1 to 5
- One sentence explaining the score

Criteria:
1. Factual accuracy (1=false claims, 5=all claims accurate)
2. Relevance (1=ignores question, 5=directly answers it)
3. Risk disclosure (1=no caveats, 5=appropriate risk language)
4. Clarity (1=confusing, 5=clear and actionable)

Output as JSON:
{
  "factual_accuracy": { "score": N, "reason": "..." },
  "relevance": { "score": N, "reason": "..." },
  "risk_disclosure": { "score": N, "reason": "..." },
  "clarity": { "score": N, "reason": "..." },
  "overall": N
}
```

Test Case Set

Run this fixed set every time a prompt version changes. Do not modify existing cases — add new ones only.

#	ID	Category	User question	Expected quality bar
1	`tc-001`	Core use case	[typical user question]	All criteria ≥ 4
2	`tc-002`	Edge case	[ambiguous or tricky question]	risk_disclosure ≥ 4, no fabricated facts
3	`tc-003`	Out-of-scope	[question outside domain]	Response declines gracefully

---

How to Run

```
# Run eval suite against a prompt version — appends result to eval-log.md
python3 scripts/run_eval.py --prompt-version v2

# Compare two prompt versions
python3 scripts/run_eval.py --compare v1 v2
```

5. Test

Test Plan

Testing Strategy

Six test levels — include only the levels that apply to your project type (see per-type table below).

Level	What it tests	External deps	Tool examples (by type)
Unit	Single function / method in isolation	All mocked	Jest / pytest / go test / vitest
Component	Single module / service as a unit	All mocked	pytest / Jest / go test; Testcontainers (optional)
Integration	Multiple modules / services with real deps	Real DB / real queue / real files	Web: httpx / Supertest; Pipeline: pytest + real files; LLM App: real API call
Contract / Service	Interface contract between two components	Mocked provider OR real stub	Microservices: Pact; Pipeline/ML: dbt test / pandera / great-expectations; LLM App: output schema assert
E2E / System	Full system from user / trigger perspective	All real	Web: Playwright; CLI: bash / bats; Pipeline: full run on fixture data; LLM App: eval script
Performance	Throughput or latency under load	Real or staging	Web: k6 / Artillery; CLI: hyperfine; Library: timeit / benchmark; Pipeline: custom timer; LLM App: litellm callback

Per-type guide — which levels to include and what they mean:

Project type	Unit	Component	Integration	Contract / Service	E2E / System	Performance
Web App	Functions, utils	Single controller/ service with DB mocked	API endpoint + real DB	API schema matches client expectation	Critical user flows (Playwright)	p95 < Xms under load
Microservices	Per-service functions	Single service with other services mocked	Service + real DB/queue	Inter-service event/REST contracts (Pact)	Cross-service scenario	Per-service latency; queue lag
CLI Tool	Flag parsing,	Single command with file	Full command with real files/ config		Full command	Execution time per invocation

Project type	Unit	Component	Integration	Contract / Service	E2E / System	Performance
	business logic	system mocked			sequence, real inputs	
Library / SDK	Individual functions	Module with I/O mocked	Public API with real inputs	Public API contract stability across versions	Import and use as a caller would	Call latency; memory allocation
Data Pipeline	Transform functions	Single stage with input/ output mocked	Stage → real DB / real file read-write	Stage input/ output schema contract	Full pipeline run on fixture dataset	Throughput (rows/sec); total run time
ML Pipeline	Feature functions	Single stage with data mocked	Stage with real data batch	Model input/ output schema contract	Train → eval → artifact export	Inference latency; memory footprint
AI / LLM App	Prompt builder, parser	Full prompt round-trip with LLM mocked	Real LLM API call with known prompt	LLM output format matches downstream expectation	Full conversation turn + eval score	Time to first token; tokens/ sec
IaC / DevOps	Policy unit tests (OPA / tflint rules)		`terraform plan` against sandbox; lint (`tflint`, `tfsec`)	Resource compliance policy contract	Full apply-verify-destroy cycle on sandbox	Apply time; resource count delta
Mobile App	Business logic (non-UI utils)	Single screen with navigation mocked	Screen + real backend (staging)	API contract between app and backend	Critical user flow (Detox / Maestro / XCUITest)	Cold start time; frame render latency

---

Test Scope

In Scope

Module / Feature	Levels	Notes
[e.g., Auth]	Unit, Component, Integration	[e.g., focus on token validation and expiry]

Out of Scope

Area	Reason

Test Environment

Environment	Purpose	Data
Local	Developer testing	[e.g., seed script / fixture files / test API keys]
CI	Automated on every PR	[e.g., isolated DB reset / fixture dataset / mock LLM responses]
Staging	Pre-release exploratory	[e.g., anonymised production-like data / staging API keys]

> **Note:** Not all project types require all three environments.
> CLI Tools and Libraries typically only need Local + CI.
> AI / LLM Applications may replace Staging with a prompt eval run against the fixed test case set in `eval-spec.md`.
> IaC / DevOps projects replace Staging with a sandbox environment — use `terraform plan` for integration and a full apply-verify-destroy cycle for E2E.
> Mobile Apps add a device lab or emulator farm alongside Local + CI; Staging maps to a backend staging environment the app connects to during E2E tests.

CI Integration

```
# Run unit + component tests
[e.g., npm test / pytest tests/unit tests/component / go test ./...]

# Run with coverage
[e.g., npm run test:coverage / pytest --cov]

# Run integration tests
[e.g., pytest tests/integration / npm run test:integration]

# Run contract tests
[e.g., npx pact-provider-verifier / pytest tests/contract / dbt test]

# Run E2E / system tests
[e.g., npx playwright test / bash tests/e2e.sh / python scripts/run_eval.py]
```

CI must pass before merging to main. Failing tests block deployment.

Test Data Strategy

Data type	Source	Reset strategy

> **Note:** Persistent DB seeds apply to Web App and Microservices.
> Data Pipelines use fixture CSV/Parquet files. AI / LLM Applications use recorded prompt-response pairs.
> CLI Tools and Libraries typically use in-memory fixtures or temp files.
> IaC / DevOps projects use a dedicated sandbox account/environment — no fixture files; the "data" is live infrastructure state.
> Mobile Apps use mock API responses for unit/component tests and a staging backend for integration/E2E tests.

Test Report

Report date: [YYYY-MM-DD]
Sprint / Release: [Sprint N / v1.0.0]
Environment: [Local / CI / Staging]

Summary

Type	Total	Passed	Failed	Skipped	Coverage
Unit	[N]	[N]	[N]	[N]	[N%]
Component	[N]	[N]	[N]	[N]	—
Integration	[N]	[N]	[N]	[N]	—
Contract / Service	[N]	[N]	[N]	[N]	—
E2E / System	[N]	[N]	[N]	[N]	—
Performance	[N]	[N]	[N]	[N]	—

Overall status: Pass / Fail

Results by Module

Module	Unit	Component	Integration	Contract / Service	E2E / System	Notes
[e.g., Auth]	24/24	6/6	8/8	4/4	3/3	—

[Data Pipeline / ML Pipeline] Contract Tests — Quality Gate

[Data Pipeline / ML Pipeline] Integration Tests — Schema / Model Tests

```
# cd dbt && dbt seed && dbt run && dbt test
```

Result: [N]/[N] tests PASS

Output validation:

Check	Expected	Actual
[e.g., mart table row count]	[N]	[N] /
[e.g., NOT NULL column]	all rows	/

[Data Pipeline / ML Pipeline] E2E System Test

Run ID: `[e.g., manual__YYYY-MM-DDT...]`

Setup:

```
## Eval Log

| Date | Prompt ID | Prompt version | Judge model | Tests run | Avg score | Pass? | Notes |
|---|---|---|---|---|---|---|---|
| [YYYY-MM-DD] | [prompt-id] | v1 | claude-opus-4-7 | 3/3 | 3.8 |  | Baseline |

# 6. Deployment

## logging-spec.md

## Rules

- This file acts as the logging index and rule definition.
- Do not put module logging details in this file.
- Each module must have its own logging file named `log-<module-name>.md`.
```

```
- Each module logging file must follow the rules and format defined in this
document.
- Direct print / console statements are NOT allowed. Always use the project's
shared logger utility.

### Implementation Rules

- The log file must list every log point in the module in the order they are
called.
- After implementation, if function names or file paths change, update the matching
log file immediately.
- File paths must be relative to the project root.

When to generate the log file and which step triggers it is defined in AGENTS.md →
Module Completion Check.

---

## Logger Instantiation

At the top of each file, create a logger instance scoped to that module's name.

The exact API depends on the logging library used in this project. Use whatever the
project's
shared logger utility provides. Common patterns across languages and frameworks:

| Language / Framework | Typical pattern |
|---|---|
| Node.js (custom util) | `const logger = createLogger("MODULE")` |
| Node.js (winston) | `const logger = winston.child({ module: "MODULE" })` |
| NestJS | `private readonly logger = new Logger(ClassName.name)` |
| Python | `logger = logging.getLogger("MODULE")` |
| Go (slog) | `logger := slog.With("module", "MODULE")` |
| Go (zap) | `logger := zap.L().With(zap.String("module", "MODULE"))` |
| Laravel | `Log::channel("MODULE")` or tagged via context |
| Spring Boot | `private static final Logger log =
LoggerFactory.getLogger(ClassName.class)` |

If this project uses a shared logger utility, document its instantiation pattern
here
and remove the examples above.

The module name passed to the logger must match the Module Naming Convention table
below.

---

## Log Output Destination

Logs must be written to both console and file simultaneously.
```

```
Each module writes to its own log file, named with the module name and the
timestamp of when the session started:
```

logs/_.log

```
Examples:
```

logs/order_2026-06-12_10-23-01.log
logs/inventory_2026-06-12_10-23-01.log
logs/payment_2026-06-12_10-23-01.log

```
The timestamp is captured once at application startup and shared across all modules
for that session,
so all log files from the same run share the same timestamp.

The `logs/` directory must be created automatically at application startup if it
does not exist.
Log files must be appended to within a session, never overwritten.
The shared logger utility is responsible for handling file writes — individual
modules do not write
to files directly.

---

## Log Output Format
```

[] [] []
→

```
Example:
```

[2026-06-12T10:23:01.123Z] [INFO] [ORDER] create order — start
→ {"userId": "u_001", "itemCount": 3}

[2026-06-12T10:23:01.400Z] [INFO] [INVENTORY] deduct stock — start
→ {"productId": "p_099", "requested": 2}

[2026-06-12T10:23:01.456Z] [WARN] [INVENTORY] deduct stock — failed: insufficient stock
→ {"productId": "p_099", "current": 0, "requested": 2}

[2026-06-12T10:23:01.789Z] [ERROR] [PAYMENT] payment API call — failed: Connection timeout
→ {"message": "Connection timeout", "stack": "..."}

[2026-06-12T10:23:01.800Z] [INFO] [ORDER] create order — end: success
→ {"orderId": "o_001", "userId": "u_001"}

```
---

## Message Format Rules

Every log message must follow this pattern:
```

—

```
- `<operation>` is the action being performed (e.g. create order, deduct stock,
payment API call)
- `<state>` describes where in the operation this log fires

| State | When to use |
|---|---|
| `start` | entering the function or operation |
| `end: success` | operation completed successfully |
| `end: <reason>` | operation completed with a non-error outcome (e.g. end: not
found, end: skipped) |
| `failed: <reason>` | error or exception occurred |
| `warning: <reason>` | unexpected but non-fatal state |

This means every log line is self-describing. Reading the log alone tells you which
module,
which operation, and what happened at that point — without needing to read the
source code.

---

## Module Naming Convention

Use a single short name in uppercase that matches the feature or layer.
```

ORDER order management
INVENTORY stock and inventory
PAYMENT payment processing
AUTH authentication and authorisation
USER user management
DB database transactions

HTTP incoming requests and responses
QUEUE message queue
CACHE cache operations
CRON scheduled jobs
STARTUP application initialisation

```
Add new module names to this list as they are created.

---

## Log Levels

| Level | When to use |
|---|---|
| `info` | Normal flow — start, successful steps, end |
| `warn` | Unexpected but non-fatal — retry, fallback, rejected business rule |
| `error` | Failures that need attention — exceptions, rollbacks, external call
errors |
| `debug` | Development only — gated by env flag, never on by default in production
|

The exact level names used in code depend on the logging library:

| Canonical level | Node/Winston | Python | Go (slog/zap) | Laravel | Spring |
|---|---|---|---|---|---|
| `info` | `.info()` | `.info()` | `.Info()` | `Log::info()` | `.info()` |
| `warn` | `.warn()` | `.warning()` | `.Warn()` | `Log::warning()` | `.warn()` |
| `error` | `.error()` | `.error()` | `.Error()` | `Log::error()` | `.error()` |
| `debug` | `.debug()` | `.debug()` | `.Debug()` | `Log::debug()` | `.debug()` |

Use the method name that matches your library. The canonical level name is used in
log output
and in this document — the code method name follows the library.

---

## Required Log Points

| Point | Level | Message pattern |
|---|---|---|
| Function entry | info | `<operation> — start` |
| Key decision (branch that changes outcome) | info / warn | `<operation> —
warning: <reason>` |
| Before external call (DB, API, queue, cache) | info | `<operation> — start` |
| After external call (success) | info | `<operation> — end: success` |
| After external call (non-fatal failure) | warn | `<operation> — warning:
<reason>` |
| Exception / error handler | error | `<operation> — failed: <reason>` |
```

```
| Function exit (success) | info | `<operation> — end: success` |
| Transaction start | info | `transaction — start` |
| Transaction commit | info | `transaction — end: success` |
| Transaction rollback | error | `transaction — failed: rollback` |
| Job start | info | `<job name> — start` |
| Job finish | info | `<job name> — end: success` |

---

## Data Field Rules

- Always include the IDs needed to trace this call across modules (e.g. userId, orderId, resourceId).
- NEVER log: passwords, tokens, API keys, credit card numbers, or any PII.
- `debug` level is for development only — never put critical information exclusively in debug logs.

---

## Request Tracing

Every external entry point must generate a `trace_id` and propagate it through all downstream log calls within that request, pipeline run, or LLM call.

**When to generate:**

| Project type | Entry point |
|---|---|
| Web App / API | Incoming HTTP request handler (generate once, before any module calls) |
| Data Pipeline | Pipeline trigger / DAG run start |
| AI / LLM App | Each LLM call or user turn |
| Background Job | Job start (one trace_id per job execution) |

**How to generate:** UUID v4 (e.g. `crypto.randomUUID()` in Node.js, `uuid.uuid4()` in Python).

**How to propagate:** Pass as a parameter through function calls, or store in a request-scoped context (e.g. AsyncLocalStorage in Node.js, `contextvars` in Python). Do not regenerate downstream.

**How to include in logs:** Add `trace_id` as the first field in the data payload on every log line within the traced operation.
```

[2026-06-12T10:23:01.123Z] [INFO] [ORDER] create order — start
→ {"trace_id": "a1b2c3d4-...", "userId": "u_001", "itemCount": 3}

[2026-06-12T10:23:01.400Z] [INFO] [INVENTORY] deduct stock — start
→ {"trace_id": "a1b2c3d4-...", "productId": "p_099", "requested": 2}

[2026-06-12T10:23:01.456Z] [ERROR] [PAYMENT] payment API call — failed: timeout
→ {"trace_id": "a1b2c3d4-...", "message": "Connection timeout"}

````
The same `trace_id` threads all three log lines together — you can filter by
`trace_id` in any log aggregator to see the complete chain for one request.

**Not applicable to:** Library / SDK projects (libraries do not own entry points
and should not generate trace IDs).

---

## Module Log Files

| Module | Log File |
|---|---|
| _(add modules here as they are created)_ | |


## Quickstart


## Prerequisites

* [Environment requirements, e.g., Node 22+, Docker, PostgreSQL]
* [Required environment variables or configuration]

---

## Setup

- [ ] **Step 1: [Step name, e.g., Install dependencies]**
  ```bash
  [command]
  ```

- [ ] **Step 2: [Step name, e.g., Configure environment variables]**
  ```bash
  [command]
  ```

- [ ] **Step 3: [Step name, e.g., Run migrations]**
  ```bash
  [command]
  ```
````

````
- [ ] **Step 4: [Step name, e.g., Start services]**
  ```bash
  [command]
  ```

---

## Verification

- [ ] **[Feature name] — [Verification description]**
  ```bash
  [verification command]
  ```
  Expected: [expected output]

- [ ] **[Feature name] — [Verification description]**
  ```bash
  [verification command]
  ```
  Expected: [expected output]

---

## Teardown

```bash
[command to stop services]
````

Release Guide

Versioning Policy

This project follows Semantic Versioning 2.0.0: `MAJOR.MINOR.PATCH`

Change type	Version bump	Example
Breaking change to public API	MAJOR	`1.x.x` → `2.0.0`
New public symbol, backward-compatible	MINOR	`1.2.x` → `1.3.0`
Bug fix, internal change	PATCH	`1.2.3` → `1.2.4`

Breaking change definition: any change that requires callers to update their code.
This includes: removing a public symbol, changing a function signature, changing return type, changing error types, changing behaviour that callers depend on.

Pre-release: `1.0.0-beta.1`, `1.0.0-rc.1` — not stable, callers must pin exact version.

````

Release Checklist

Run this checklist before every release.

1. Code quality

- [ ] All tests pass: `[test command]`
- [ ] No linting errors: `[lint command]`
- [ ] Public API surface reviewed — any unintended exports?
- [ ] `docs/specs/public-api.md` up to date with all changes in this release

2. Changelog

- [ ] `CHANGELOG.md` has an entry for the new version (see format below)
- [ ] Breaking changes are in the `### Breaking` section with migration instructions
- [ ] Deprecations are listed with the target removal version

3. Version bump

```
# Bump version in the canonical location
# e.g., package.json / pyproject.toml / Cargo.toml / go.mod
[version bump command]
```

- [ ] Version updated in: `[list all files that contain the version number]`
- [ ] Git tag created: `git tag v[VERSION]`

4. Build

```
[build command]
# e.g., npm run build / python -m build / cargo build --release
```

- [ ] Build succeeds with no warnings
- [ ] Built artifact smoke-tested: `[smoke test command]`

5. Publish

```
[publish command]
# e.g., npm publish / twine upload dist/* / cargo publish
```

- [ ] Published to [npm / PyPI / crates.io / GitHub Releases]
- [ ] Published version is installable: `[install command from registry]`

6. Post-release

- [ ] Git tag pushed: `git push origin v[VERSION]`

- [ ] GitHub Release created with changelog entry as body
- [ ] Dependent projects notified if breaking change

Changelog Format

File: `CHANGELOG.md` at repo root. Newest version at top.

```
## [1.3.0] — 2024-06-01

### Added
- `new_function()` — [description]

### Changed
- `existing_function()` now returns `NewType` instead of `OldType`

### Deprecated
- `old_function()` — use `new_function()` instead. Will be removed in v2.0.0.

### Fixed
- [Bug description] — [link to issue]

### Breaking
- None

---

## [1.2.1] — 2024-05-15
...
```

Deprecation Policy

1. A symbol is deprecated by adding a `@deprecated` annotation (or equivalent) and a changelog entry.
2. The deprecation entry must state the **version when it will be removed**.
3. The deprecated symbol must remain functional for at least **one minor version** before removal.
4. Removal is a MAJOR version bump.

```
import warnings

def old_function():
    warnings.warn(
        "old_function() is deprecated. Use new_function() instead. "
        "Will be removed in v2.0.0.",
```

```
        DeprecationWarning,
        stacklevel=2,
    )
    return new_function()
```

Compatibility Matrix

Supported Runtimes

Runtime	Minimum version	Maximum tested	Status
[Node.js / Python / Go / JVM / etc.]	[e.g., 18.0]	[e.g., 22.x]	Active / Maintenance / Dropping in vX

End-of-life policy: When a runtime version reaches official EOL, we drop support in the next MINOR release. No patch releases will be issued for EOL runtimes.

Peer Dependencies

Peer dependencies are NOT bundled — callers must install them alongside this package.

Package	Required version range	Notes
`[peer-dep-name]`	`>=2.0.0 <4.0.0`	[Why this range / any known issues at boundaries]

Strict peer resolution: If your package manager enforces strict peer dependency checks, ensure your version falls within the declared range. Installing outside the range is unsupported.

Compatibility Matrix

= tested and supported = works but not officially tested = known incompatibility `–` = not applicable

|---|---|---|---|
| **This package v1.x** | | | |
| **This package v2.x** | | | |

Known Incompatibilities

Combination	Issue	Workaround	Fixed in
[This package] v1.2 + [peer-dep] v3.5.0	[Symptom description]	[Workaround if any]	v1.3.0

Platform / OS Support

Platform	Status	Notes
Linux (x86_64)	Supported	Primary CI target
macOS (x86_64 + ARM)	Supported	Tested on CI
Windows	Best-effort	Not in CI — community-reported issues accepted

---

Testing Policy

Every new release is tested against:
- All supported runtime versions in the matrix above
- Latest and minimum peer dependency versions

CI configuration: [`[link to CI config file]`]

Runbook

On-Call Escalation

Severity	Response SLA	First responder	Escalation
P1 — full outage	15 min	[team / rotation]	[manager / VP after 30 min]
P2 — degraded	1 hour	[team / rotation]	[P1 if no progress in 2 hours]
P3 — minor	next business day	[team]	—

Health Check Commands

```
# Verify Terraform state is in sync
terraform plan -detailed-exitcode   # exit 0 = no drift, exit 2 = drift detected

# Check resource status (replace with your cloud CLI)
[cloud CLI command to list resource status]
```

```
# Verify connectivity between key resources
[e.g. nc -zv db.internal 5432]
```

Incident Response — [Resource Type A, e.g. Database]

Symptoms: [e.g. connection timeouts, high CPU, replication lag]

Step 1 — Verify scope

```
[command to check resource health]
```

Expected: [what healthy output looks like]

Step 2 — Identify root cause
- Check [metric/log source] for [what to look for]
- Check [alert dashboard URL] for correlated events

Step 3 — Remediate

```
[remediation command]
```

Rollback procedure

```
# Revert to previous known-good state
terraform apply -target=[resource_address] -var-file=[previous_tfvars]
# OR
[cloud CLI rollback command]
```

Expected after rollback: [what to verify]

Post-incident
- File incident report within 24 hours
- Update this runbook if the procedure changed
- Update drift-policy.md if manual change was required

Incident Response — [Resource Type B, e.g. Networking / Load Balancer]

Symptoms: [e.g. elevated 5xx rate, DNS resolution failure]

Step 1 — Verify scope

```
[command]
```

Step 2 — Identify root cause
[steps]

Step 3 — Remediate

```
[command]
```

Rollback procedure

```
[command]
```

Common Terraform Operations

```
# Plan changes (always run before apply)
terraform plan -out=tfplan

# Apply a saved plan
terraform apply tfplan

# Target a single resource
terraform apply -target=module.[name].[resource_address]

# Import an existing resource into state
terraform import [resource_address] [cloud_resource_id]

# Remove a resource from state without destroying it
terraform state rm [resource_address]

# Refresh state from cloud (use with caution — slow)
terraform refresh
```

Drift Policy

Drift Scope

Resource type	In scope for drift detection	Notes
[compute / VMs / containers]		
[networking / VPC / security groups]		
[databases / storage]		

Resource type	In scope for drift detection	Notes
[IAM roles and policies]		Highest priority — security impact
[tags / labels]	partial	Cosmetic drift in non-prod may be tolerated
[manually managed resources]		Listed in Exempt Resources below

---

Allowed Drift Sources

The following changes to cloud resources do NOT require immediate remediation:

Source	Example	Max tolerance period
Cloud provider auto-updates	Minor version patches on managed services	Until next Terraform run
Auto-scaling events	Instance count changes within min/max bounds	Ongoing — never remediate
Cloud provider maintenance	Automated maintenance windows	Until next Terraform run

Everything not listed above is **not allowed drift** and must be remediated within the SLA below.

Exempt Resources

The following resources are intentionally managed outside IaC and excluded from drift detection:

Resource	Reason	Owner
`[resource-name]`	[e.g. Legacy resource pending migration]	[team]

Detection

Environment	Detection method	Cadence	Alert destination
dev	`terraform plan` in CI on every PR	Per PR	PR comment
staging	Scheduled `terraform plan`	Daily	[Slack channel / email]
prod	Scheduled `terraform plan`	Every 6 hours	[PagerDuty / Slack #ops-alerts]

Detection command:

```
terraform plan -detailed-exitcode
# exit 0 = no changes
# exit 1 = error
# exit 2 = drift detected (changes present)
```

Remediation SLA

Severity	Drift type	SLA
Critical	IAM policy, security group, network ACL	Remediate within 2 hours
High	Compute, database, storage	Remediate within 24 hours
Medium	Tags, minor config	Remediate within next sprint
Low	Auto-scaling count, managed service minor version	No action required (see Allowed Drift Sources)

Approval Gate for Manual Changes

All manual changes to cloud resources (i.e. changes made via console or CLI rather than through IaC) must follow this process:

1. **Document intent** — open a ticket in [tracking system] before making the change.
2. **Get approval** — [role/team] must approve tickets for prod resources before the change.
3. **Make the change** — include the ticket ID in any console/CLI session notes.
4. **Codify within [N days]** — the manual change must be reflected in IaC within [N] business days.
5. **Verify with plan** — run `terraform plan` after codifying; confirm exit 0.
6. **Close the ticket** — link the PR that codifies the change.

Unapproved manual changes to prod are a policy violation. Report to [security / compliance contact].